

Music Generation with Generative Diffusion Models

Sebastião Correia

*Master of Science in Computer Vision, Machine Learning and
Robotics*

from the

University of Surrey



Department of Electronic Engineering

Faculty of Engineering and Physical Sciences

University of Surrey

Guildford, Surrey, GU2 7XH, UK

September 2024

Supervised by: Xiatian Zhu

©Sebastião Correia 2024

DECLARATION OF ORIGINALITY

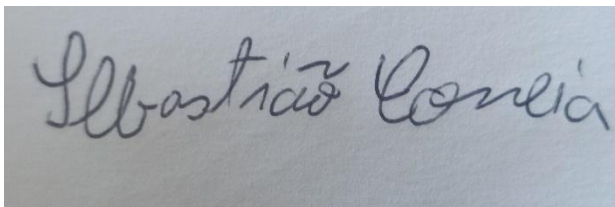
I confirm that the project dissertation I am submitting is entirely my own work and that any material used from other sources has been clearly identified and properly acknowledged and referenced. In submitting this final version of my report to the JISC anti-plagiarism software resource, I confirm that my work does not contravene the university regulations on plagiarism as described in the Student Handbook. In so doing I also acknowledge that I may be held to account for any particular instances of uncited work detected by the JISC anti-plagiarism software, or as may be found by the project examiner or project organiser. I also understand that if an allegation of plagiarism is upheld via an Academic Misconduct Hearing, then I may forfeit any credit for this module or a more severe penalty may be agreed.

Music Generation with Generative Diffusion

Sebastião Correia URN 6578555

Author Signature

Date: 22/12/2024

A photograph of a handwritten signature in dark ink on a light-colored surface. The signature is written in a cursive style and reads "Sebastião Correia".

Supervisor's name:

WORD COUNT

Number of Pages: 71

Number of Words: 25266

ABSTRACT

AI Music generation has undergone significant advancements with the advent of artificial intelligence. Recent advancements in generative machine learning algorithms have made it a more feasible endeavour. Allowing it to capture the very nuanced and complex nature of audio and music leading to a higher quality in generation. Diffusion models have been shown to be quite remarkable in high quality image generation. They have become quite popular in their use in applications such as DALL E, MidJourney, etc. In this thesis I explore the use of diffusion models to generate music conditionally and unconditionally, in particular by transforming music samples into Mel spectrogram representations and using them to train Diffusion models.

TABLE OF CONTENTS

Declaration of originality	ii
Word Count	iii
Abstract	iv
Table of Contents	v
1 Introduction.....	2
1.1 Background and Context.....	2
1.2 Scope and Objectives	3
2 BACKGROUND THEORY AND LITERATURE REVIEW	3
2.1 Audio Representation.....	3
2.1.1 Waveform.....	3
2.1.2 Spectrograms.....	4
3 Generative machine learning models for sound synthesis	5
3.1 Early generative models.....	6
3.2 Recurrent Neural Networks (RNNs) and Long Short-Term Memory (LSTM) Networks	6
3.3 Autoencoders	7
3.3.1 Autoencoders in Audio and Music Synthesis	9
3.3.1.1 Musical Audio Synthesis using Autoencoding Neural Nets [24]:	9
3.3.1.2 Improving Neural Net Autoencoders for Music Synthesis [25]:.....	9
3.3.1.3 Neural Audio Synthesis of Musical Notes with WaveNet Autoencoders [27]:	10
3.3.1.4 Autoencoders for music sound modelling [28]:	10
3.4 Variational Autoencoders (VAE).....	12
3.4.1 The ELBO.....	13
3.4.2 Reparameterization trick.....	15
3.4.3 The β -VAE	17
3.4.4 RAVE: A variational autoencoder for fast high-quality neural audio synthesis [34]:	17
3.5 Vector-Quantized Variational Autoencoder (VQ-VAE)	19
3.5.1 VQ-VAE 2 [43]:.....	20
3.5.2 Open AI's Jukebox [5]:	22
3.6 Generative Adversarial Networks (GANs)	24
3.6.1 Jensen-Shannon Divergence	25
3.6.2 Instability in GANs	26

3.6.3	Wasserstein GAN.....	26
3.6.4	WaveGAN [3]:	27
3.6.5	Parallel WaveGAN [52]:	29
3.6.6	MelGAN [41]:.....	30
3.7	Diffusion Models	31
3.7.1	Introduction.....	31
3.7.2	Denoising Diffusion Probabilistic Models (DDPMs)	32
3.7.3	Improved DDPM	36
3.7.4	Denoising Diffusion Implicit Models (DDIM)	38
3.7.5	DiffWave:.....	41
3.7.6	WaveGrad:	41
3.7.7	Noise2Music[7]:	41
3.8	Evaluation Methodology.....	42
3.8.1	Log-Likelihood	42
3.8.2	Inception Score	43
3.8.3	Fréchet Inception Distance	45
3.8.4	Fréchet Audio Distance.....	45
3.8.5	Human Assessment	47
4	Implementation	47
4.1	Audio Representation.....	48
4.2	Datasets	48
4.3	Architecture.....	49
4.3.1	Blocks	49
4.4	Training.....	50
4.4.1	Training Setup.....	50
4.4.2	The Training Script	51
4.5	Evaluation	51
4.5.1	Latent space sampling.....	51
4.5.2	Reconstruction	52
4.5.2.1	Individual Reconstruction	52
4.5.2.2	Reconstruction Evaluation	53
4.5.3	Interpolation.....	53
4.5.4	Fréchet Audio Distance (FAD)	54
4.5.4.1	FAD Implementation.....	55
5	Experimentation and Results	56
5.1	Quantitative evaluation	56
5.2	Generation.....	58

5.3	Reconstruction	58
5.4	Interpolation	59
6	Conclusion	61
6.1	Future Work	61
	References.....	62
	Appendix.....	68
A.1	Detailed Mathematical Concepts and derivations	68
A.1.1	Kullback-Leibler Divergence.....	68
A.2.1	Expectation of a random variable	68
	List of Figures	
Figure 1		8
Figure 2	Taken from [33]	15
Figure 3:	From [41], VQ-VAE architecture on the left, Embedding space visualization of the right.	19
Figure 4	Taken from [17]	35
Figure 5:	Taken from [62]: Comparison of different divergence measures when fitting a Gaussian distribution to a mixture of Gaussians.	43

1 INTRODUCTION

Recent advancements in generative artificial intelligence have been groundbreaking providing ever-increasing sophistication most famously in image generation, speech synthesis, large language models, etc. However, one area that has not received as much attention, is non-speech audio generation, such as AI-generated music. Fortunately, this domain is undergoing significant innovation with the emergence of new and improved deep generative neural network techniques. You can access my code in [sebascorreia/Music-Diffusion \(github.com\)](https://github.com/sebascorreia/Music-Diffusion) with the colab notebook being under the notebook folder as. My models can be found at [sebascorreia \(Sebastiao Correia\) \(huggingface.co\)](https://huggingface.co/sebascorreia)

1.1 Background and Context

AI-generated music can be traced back to the 50s and 60s with the first computer generated composition being created by Lejaren Hiller et al using the Illiac I computer in 1957. This was a basic symbolic music generation using Markov chains and preset rules. For a while music generation was confined to symbolic music generation, with rule-based systems and the appearance of neural network compositions in the late 80s and 90s, RNNs, LSTMs, to more recent Midi and complex melody compositions using autoencoders and GANs.[1]

AI driven audio synthesis began with the purpose of replicating human speech, starting with basic techniques by todays standards such as linear predictive coding in the 60s, moving on to Hidden Markov models around the 80 and 90s, moving on to deep learning techniques by the 2000s. The advancements in these deep learning techniques and better GPU capabilities allowed for development of new techniques to model the actual waveform of an audio.

A pioneer of such developments was WaveNet. This model demonstrated the ability to directly model raw audio waveforms with significant sound quality, able to replicate its intricate and complex details. This showed potential in synthesising speech, but also in synthesizing music, with both conditional and unconditional generation. Unconditional generation means that a model was trained on audio data without any guidance of labels or other types of conditioning data. The unconditional generation of Wavenet although not very consistent in the long term, with frequent changes in instrument, sound quality, genre, and volume, it was able to generate harmonic and aesthetically pleasing samples. Conditional generation means the models are trained with audio data along with associated conditional information. This conditional information can be lyrics, speaker id, bird chirp species name, song genre, instruments, etc. This allows the model to be able to conditioned on genre or instrument labels, and although they were much more consistent in genre and instrument the results were nevertheless very erratic, with the melody and tempo constantly shifting almost every second.[2] After WaveNet, researchers experimented with newer and more advanced

generative machine learning approaches such as generative adversarial networks (GANs) such as WaveGAN [3], variational auto-encoders and transformers such as Jukebox [5], language modeling such as AudioLM[6], and Diffusion models such as Noise2Music [7].

1.2 Scope and Objectives

This dissertation aims to explore non-autoregressive generative models and their use in the context of non-symbolic audio synthesis, and to implement diffusion models for both conditional and unconditional mel-spectrogram audio generation.

2 BACKGROUND THEORY AND LITERATURE REVIEW

2.1 Audio Representation

WaveGAN and SpecGan were the first applications of Generative adversarial networks to generate raw audio and audio spectrograms in an unsupervised approach. The results showed promise for both wave and spectrogram-based approaches to audio generation, with spectrogram being able to capture the variance in the underlying data better than WaveGAN but having worse sample quality caused by the lossy Griffin-Lim algorithm for spectrogram inversion. [3]

2.1.1 Waveform

Analog sound is produced by changes in pressure caused by vibrations from various sources, such as vocal cords, musical instruments, etc. which create waves in the medium the sound is traveling, such as air. Those waves can be captured by microphones and converted into electrical signal which can in turn be sampled into a digital signal representation. This digital signal represents the wave as a sequence of numbers, each number being an amplitude sample at a chosen sampling frequency. To capture the all the necessary information to accurately represent that sound the sampling frequency should adhere to the Nyquist-Shannon sampling theorem. This theorem states that only frequencies lower than half of the sampling frequency can be accurately reproduced from a sampled signal. [8]

The WAV audio file format is the main format used for uncompressed audio signal. These are usually composed of two channels for stereo audio, with a sampling frequency of 44.1kHz. Meaning per second of stereo audio would have a tensor shape of (44100x2). Comparatively a standard image of size (256x256x3) corresponds to about 2 seconds of audio. While this form preserves the complete and accurate representation of the original sound, without the need for lossy transformations and intermediate representations, the complexity and the sheer amount of data makes it more computationally and memory expensive for longer sequences, making it very time consuming

to directly work with.

As a result, the raw audio is frequently transformed into more concise and meaningful representations, this can be by up-sampling, feature extraction, or different representations altogether such as spectrograms and music notation.

2.1.2 Spectrograms

The spectrogram represents the sound in a time/frequency visual representation. It is produced by a Short-Time Fourier-Transform (STFT). The STFT takes Fourier transform at short segments of the wave decomposing the complex waveform into its constituent sine and cosine waves. This allows for a detailed examination of the frequency composition of a sound over time. The x axis represents time, the y axis represents frequency, and the colour intensity represents amplitude. Each column of the spectrogram represents a small segment of the waveform. STFT of the signal x at frequency f and time l is defined as:

$$\begin{aligned}
 STFT\{x\}(f, l) &= X(f, l) : \\
 &= \sum_{t=0}^T x_t * w_{t-l} * e^{\left(-i\frac{2\pi t}{T}f\right)} \\
 &= \sum_{t=0}^T x_t * w_{t-l} * \left(\cos\left(\frac{2\pi t}{T}f\right) - i \sin\left(\frac{2\pi t}{T}f\right)\right)
 \end{aligned} \tag{1}$$

Where w is a window function that isolates a short segment of the waveform around time l . Common choices for w include the Hamming window, Hann window, Gaussian window, among others. These are mathematical functions that are designed to attenuate the signal gradually as it moves away from the centre of the window. The specific characteristics of the chosen window function can significantly influence the spectrogram's resolution, dictating its capacity to accurately represent rapid changes (transients) and sharp discontinuities in the underlying signal. An important aspect to consider here is the trade-off between time and frequency resolution; a wider window offers better frequency resolution but worse time resolution and vice versa.

Using Euler's formula $e^{ix} = \cos(x) - i \sin(x)$ the complex exponential function $e^{\left(-i\frac{2\pi t}{T}f\right)}$ is broken down to its sine and cosine components. Where t is the time index, f is the frequency variable, T is the total duration or number of samples in a signal. The cosine and sine terms essentially correlate the signal with cosine and sine waves of frequency f , respectively. This allows us to represent signals in a complex plane, where the cosine component gives the real part, and the sine component gives the imaginary part. These real and imaginary parts can be converted into magnitude and phase. This is crucial to determine the amplitude of each frequency component in the

signal by plotting a magnitude spectrogram.

Paired with the magnitude we also can get the phase information which is crucial to fully reconstruct the original signal back to the real and imaginary parts and back to its original audio. However, in many applications, the phase information is discarded, as it is very hard to compress compared to the magnitude information, due to its complex nature almost looking like random noise. Although, neglecting phase information can result in a loss of temporal structure in a signal. Magnitude only spectrograms have the advantage of being more easily compressible, but the disadvantage of being harder to decode.

To adjust to the loss of phase information, many choose to train additional models called vocoders that predict the phase or directly predict the waveform from a magnitude spectrogram. There are also phase estimation algorithms used to recover phase such as the Griffin-Lim algorithm, Phase-gradient heap integration, single-pass spectrogram inversion, etc. These have different trade-offs and must be tuned to best find the optimal parameters. [9]

In order to reduce unnecessary information and provide a more perceptually relevant representation of a sound signal, we can apply the Mel scale to the magnitude spectrogram to make it more in tune with the non-linear human perception of sound. The Mel scale is given by the expression: $2595 \log_{10} \left(1 + \frac{f}{700} \right)$. This emphasizes certain frequency bands based on the characteristics of human hearing. This has the advantage of making a model focus on the more relevant details as perceived by the human auditory system. Though it has the disadvantage of not being invertible back to the linear scale without loss of information. This is because the Mel scale groups frequencies into bands, reducing the resolution of the spectrogram as many frequencies will lie on the same band [10]. This is paired with the loss of the phase information. To alleviate this loss of information, optimization-based methods can be used, alternatively you could also use vocoders trained to act as scale inverter and phase reconstructor in one.[4]

3 GENERATIVE MACHINE LEARNING MODELS FOR SOUND SYNTHESIS

Generative machine learning models are paving the way for innovative approaches to music creation and manipulation. These models are designed to generate new data samples that are similar to a given set of training data. In the context of music generation, they take existing samples of music data as inputs and produce new music samples that retain essential characteristics of the original data. They operate by learning the underlying distributions and intricate patterns hidden within the music data during a training phase, then use this learned knowledge to generate new data that follows similar distributions and patterns in the inference phase. From the production of background

scores for films [32], to aiding composers in developing unique sound patterns, generative models are becoming increasingly mainstream, and a highly sought-after tool in the modern musical toolkit.

3.1 Early generative models

Before the advent of deep learning-based generative models, early statistical methods such as Hidden Markov Models (HMMs) and Bayesian Networks played a key role in the field of music generation.

HMMs emerged as a valuable tool in modelling sequential data. In the context of music generation HMMs could be utilized to analyse the sequence of musical notes or chords, modelling the statistical relationships between different elements in a piece of music. By understanding the probabilities of transitioning from one musical state to another, HMMs could create cohesive musical sequences that maintained the structural integrity of the original compositions. [11]

Bayesian Networks have surfaced as powerful tools in modelling both simultaneity and sequentially in music generation. By understanding the probabilistic relationships between different elements of music such as pitch, rhythm and tempo, these networks can effectively learn the complex interdependencies among them. This is useful in applications such as chord voicing, four-part harmonization, and real-time chord prediction, allowing for a unified approach to generating well-structured and harmonious music. [12]

Both HMMs and Bayesian Networks laid the groundwork for generative algorithms used for generating music, although generating music in a symbolic representation. While they may be somewhat overshadowed by more recent deep learning methods, these early models still hold historic significance, offering unique perspectives in the evolving landscape of automated music generation.

3.2 Recurrent Neural Networks (RNNs) and Long Short-Term Memory (LSTM) Networks

In the early stages of deep learning, research turned to RNNs to effectively process sequential and time-series data as the usual Convolutional or Deep neural networks could not do so. Therefore, in many research areas that contain sequential data such as text, audio, and video, RNNs became dominant. The RNN's cyclic connections allow it to update the current state based on past states and current input data. This creates a kind of memory which can capture information about what has been calculated so far. Unfortunately, when there is a large gap between relevant input data, traditional RNNs struggle to link the relevant information effectively.

To deal with these long-term dependencies the long short-term memory (LSTM) network was

proposed by Hochreiter and Schmidhuber [13]. The LSTM was able to capture longer dependencies and avoid the vanishing gradient problem which was a common problem with RNNs where information could be lost over many time steps during training. The LSTM accomplishes this through a series of gates that regulate the flow of information to be remembered or forgotten at each time step, maintaining a delicate balance between retaining valuable historical data and accommodating new potentially important inputs. [14]

LSTM networks have been shown to be a remarkable tool as generative models. Being used to create new content, be it in text, music, speech synthesis and other sequential data. In music generation they have proven themselves to be a powerful tool capable of learning harmonic and melodic note sequences and generating polyphonic midi compositions.[15]

3.3 Autoencoders

Though not primarily used for generating new unseen data Autoencoders can provide great insights on more complex generative models such as variational autoencoders and can be seen as an entry way to understanding the latent space. Autoencoders are designed to learn an informative lower-dimensional representation of the data that can be used for various applications. This is achieved by training a model to learn key features and accurately reconstruct a set of input observations.[20] We can separate this method into two functions the encoder (g) and the decoder (f). Both neural networks which can be defined as:

$$z = g(x) \quad (2)$$

$$\tilde{x} = f(z) = f(g(x)) \quad (3)$$

Where x are the input observations, z are the latent representations and $\tilde{x}$ are the reconstructed input observations. [21] The encoder maps a high-dimensional input x into a low-dimensional latent vector z , capturing a meaningful representation of the input vector. The decoder takes that low-dimensional representation z and reconstructs its best estimate of the input back into its original dimensions. The loss function for this approach can be written as:

$$\operatorname{argmin}_{f,g} \mathbb{E} \left[\Delta \left(x, f(g(x)) \right) \right] \quad (4)$$

Where Δ is the difference between the input and its reconstruction, usually done with mean squared error or binary cross-entropy. $\mathbb{E}$ is the expectation which averages the reconstruction error across all data instances. In other words, are trying to minimize the average of the reconstruction error across all data instances.

A fundamental problem when it comes to autoencoders is the Bias-Variance trade-off. This is the balance between the model being able to generalize(bias) and its sensitivity to specific data (vari-

ance). High bias leads to the autoencoder missing key features of the data, leading to poor reconstruction. High variance leads to overfitting to the training data, leading to poor performance on unseen data. The goal is to find a sweet spot where the autoencoder accurately captures the key features of the data without overfitting, guaranteeing an effective and generalized data representation and reconstruction. [20]

To avoid the encoder and decoder becoming identity operators and to find the bias-variance sweet spot, we need to implement regularization. This is usually done with bottlenecks in the architecture but can also be done by enforcing sparsity to the neuron activations, or by doing both. Implementing a bottleneck in the autoencoder architecture means lowering the dimension of the latent representation to something smaller than the original input's and the reconstruction's dimensions. An example of a bottleneck architecture could be:

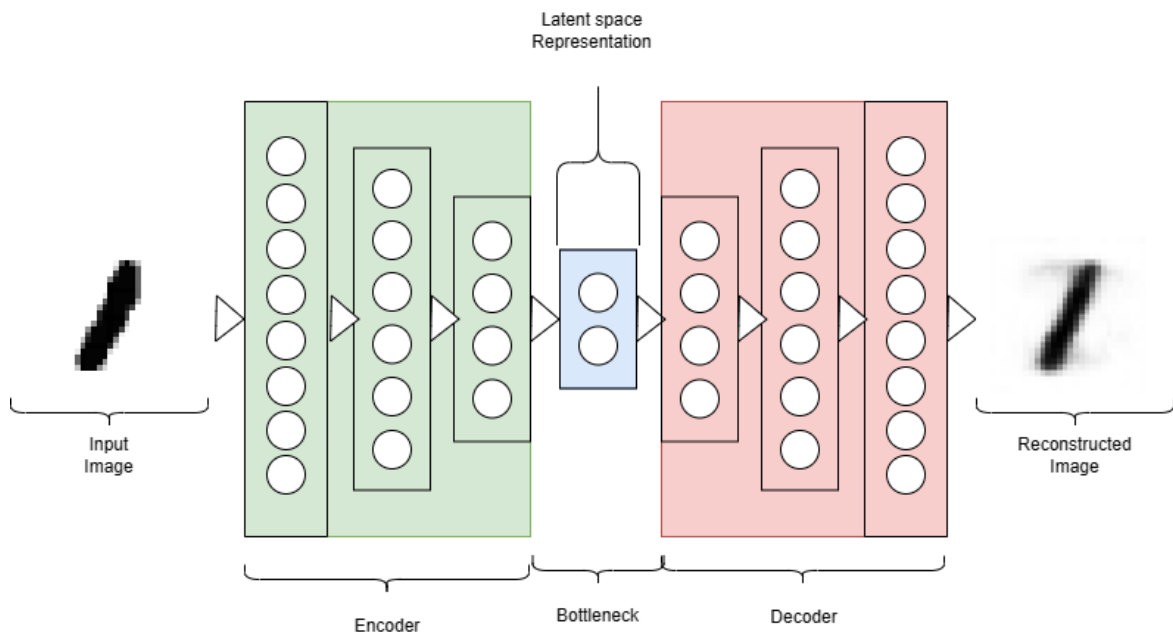


Figure 1

Where the white circles represent the neurons, you can see that the encoder's dimensionality decreases until it reaches the bottleneck. Then it increases back to its original dimensionality in the decoder. This forces the autoencoder to learn a different representation of the input and not simply an identity function.

To enforce sparsity means to ensure a significant portion of the neuron activations become inactive or have a near-zero activation. This means only a small number of neurons are active at any given time. Much like the bottleneck this ensures the autoencoder doesn't just learn to copy its input to its output without capturing useful features. Sparsity regularization can be used on its own or in addition to a bottleneck architecture, to provide further regularization.

There are a couple of ways to do it, L1 regularization or KL-Divergence, applied to the activation

of the neurons. The first method is to use the L1 norm to penalize non-zero values pushing many neuron activations towards zero. This means our objective function will look like:

$$\operatorname{argmin}_{f,g} \mathbb{E} \left[\Delta \left(x_i, f(g(x_i)) \right) \right] + \lambda \sum_i |a_i| \quad (5)$$

Where a_i is the activation at the i th hidden layer and i iterates over all hidden activations. [20]

λ is the regularization parameter used to control the strength of the sparsity enforcement.

3.3.1 Autoencoders in Audio and Music Synthesis

3.3.1.1 Musical Audio Synthesis using Autoencoding Neural Nets [24]:

Autoencoders have been applied in the field of music synthesis. One of the pioneering works in this subject being the work of Sarrof et al in [24] which utilized a shallow autoencoder and a more complex architecture composed of two stacked shallow autoencoders trained to reconstruct magnitude Fourier transforms from a dataset comprising 70,000 randomly selected frames across 8,000 songs. In this work a system was developed for users to manipulate the autoencoder model's parameters to change the synthesis in real time. This study had the shortcoming of high reconstruction error in the models, predominantly in the deeper architectures, impacting the quality of the audio synthesis.

3.3.1.2 Improving Neural Net Autoencoders for Music Synthesis [25]:

Colonel et al in [25] improved on the work done on [24] by improving regularization, creating a better dataset, choosing more appropriate activation functions, and implementing the Adam learning method. They improved regularization, replacing the denoising techniques which they deemed unnecessary. Using dropout to encourage robustness, improving the quantitative performance of the autoencoders. This has the downside of limiting expressiveness, due to ignoring small changes in the latent space. They also used l2 regularization which encourages the model to use smaller weights, this improved the qualitative performance of the autoencoder by improving model convergence.

In their exploration of activation functions, they found using all sigmoid functions resulted in vanishing gradient, to fix this they used ReLU in all layers except for the hidden and output layers. They found that only using all ReLU would produce an acceptable MSE but the output signal sounded tinny. Their choice for a mixed sigmoid and ReLU topology allowed for the multiplicative gains used for audio modulation to take an effect on the latent representation, in the layers using sigmoid activation. The sigmoid activation function was also an appropriate choice for the output layer as the magnitude STFT frames have a range of 0 to 1, which is the same range of values produced by the sigmoid function.

The Adam training method provided much better MSE results compared to the momentum-based training method used in [24], this suggests that the momentum-based training method got caught in poor local minima. While the Adam method provided adaptive properties which allowed the models to search the weight space for more robust minima.

The authors also chose a dataset that was more focused on tonal sounds instead of a dataset with several genres as used in [24]. This dataset was composed of 70,000 magnitude Fourier frequency transform frames 10k of which was for testing and another 10k for validation. Generated by a MicroKORG synthesizer, this restricted the autoencoder to a harmonic synthesizer with a maximum of four note polyphony, so it did not have to learn to reconstruct more complex features such as rhythm, speech, and complex harmonic tones.

3.3.1.3 Neural Audio Synthesis of Musical Notes with WaveNet Autoencoders [27]:

This study done by Engel et al [27] presents an autoencoder inspired by WaveNet [2]. While WaveNet proposed external conditioning based on “temporally aligned linguistic features”, this autoencoder removes the need for external conditioning. Uses a temporal encoder to infer embeddings distributed in time. These embeddings guide the WaveNet based decoder to autoregressively generate audio waveforms one sample at a time. This decoder is conditioned by adding a linear projection of the temporal embeddings as a bias at every layer, effectively shaping the waveform generation. The embeddings capture local context, allowing the model to maintain long-term structure and to generalize in time. This means the model can reconstruct notes played for longer than 3 seconds and can even reconstruct note sequences such as arpeggios adequately despite never actually having seen a transition between two notes. This model also achieved interesting results in inter instrument interpolation. Whilst the interpolation in the original audio-space are simple superpositions of each note, the WaveNet autoencoder fuses both notes into one, creating a distinct note from the original notes by merging aspects of the timbre and dynamics of the two instruments.

In this paper they presented a new large-scale dataset called NSynth used to train the Wavenet autoencoder and the baseline comparison autoencoder model. It is composed of 306,043 unique musical notes, from 1006 instruments. Each entry is made up of 4 seconds of monophonic audio containing unique pitch, timbre and envelope. This dataset was created to fill the need for a public large-scale audio dataset comparable to those in the image domain.

3.3.1.4 Autoencoders for music sound modelling [28]:

Roche et al presents a study investigating different autoencoder architectures and their ability to compress music datasets to latent spaces and subsequently reconstructing and synthesizing music from those latent representations. The study has three objectives: i) to compare shallow (AE), deep (DAE), Variational (VAE), and Long Short-Term Memory (LSTM-AE) Autoencoders with a base-

line Principal Component Analysis (PCA), using Root Mean Square Error (RMSE) as an objective physical measure and PEMO-Q as an objective perceptual measure to compare the efficacy of the reconstruction and sound quality of synthesis; ii) to evaluate the characteristics of the latent space by analysing the correlation among its dimensions; iii) to generate interesting hybrid sounds using interpolation in the latent space.

The study used the previously mentioned NSynth dataset [27] to train the models using 10,000 different randomly chosen sounds from all varieties of instruments, pitches and velocities. Short-time Fourier transform (STFT) is used to extract the magnitude and phase short-term spectra, the magnitude spectra is the input of the encoders, and the phase spectra is stored for later use when synthesizing the output audio. The encoders output a latent representation of the magnitude spectra which is fed into the decoder after some possible modifications to reconstruct the spectra. Without any modification of the latent coefficients the reconstructed magnitude spectra can be synthesized with the phase spectra by applying Inverse STFT. If it is modified, then the Griffin-Lim Algorithm must be used to estimate the phase spectra that fits the modified magnitude spectra.

The study explores various autoencoder architectures, starting with a shallow autoencoder (AE) comprising a single encoder and decoder layer. Interestingly, the AE showed a higher reconstruction error than the baseline PCA contradicting results found in related work within the image processing domain. The Deep Autoencoder (DAE) can be created by stacking shallow autoencoders or adding more encoder and decoder layers. The DAE can be trained layer-by-layer or end-to-end, although no significant difference in performance was found between the two. DAE notably outperformed both PCA and shallow AE in reconstruction accuracy, achieving up to 20% improvement in RMSE, particularly in configurations with lower-dimensional latent spaces. The Long Short-Term Memory Autoencoder (LSTM-AE) has roughly the same architecture as the AE but uses LSTM cells in place of the traditional neural network neurons. These LSTM cells are adept at recognizing and encoding temporal dependencies within the data as they consider prior input observations over extended periods. This is particularly useful for tasks involving time-series data such as speech and music, where it is crucial to capture a sense of continuity and sequence of the input. The LSTM-AE had a significantly lower RMSE than the shallow autoencoder and the PCA, and slightly lower than the DAE. I will discuss the Variational Autoencoder (VAE) in the next chapter.

The evaluation of the audio quality was conducted using PEMO-Q scores. These demonstrated a strong correlation between PEMO-Q and RMSE, with the notable exception of the LSTM-AE. The PCA and AE exhibited almost identical PEMO-Q scores, with PCA marginally on top. DAE and VAE outperformed PCA, showing improvements up to 11% in some of the lower latent space dimensions. Interestingly, despite outperforming PCA in the lower encoding dimensions, the LSTM-AE's PEMO-Q scores declined with the increase of encoding dimension, being surpassed by PCA

around 50 encoding dimensions. It is important to note that excellent reconstruction accuracy, as indicated by RMSE, does not necessarily translate to high audio quality, bringing to light the intricate and subtle nuances of audio perception.

This study explores the potential for the latent spaces generated by each model to be utilized as musician-controllable parameters. They emphasized the need for these parameters i) to be as uncorrelated as possible, to minimize redundancy in the spectrum encoding and ii) to have clear and easily interpretable perceptual meaning. To study the first constraint, they compared the correlation matrices of the latent spaces for PCA, DAE, LSTM-AE and three VAEs with different beta values. The study concluded that the LSTM-AE and DAE latent spaces had the most correlated dimensions, this is due to the complexity of their nature and when comparing to VAEs because of the properties of VAEs which will be discussed in the next chapter. Such properties of variational autoencoders allow the model to have both less correlated dimensions and to limit the latent space making it a better model for extracting good control parameters and to have a fairly good reconstruction accuracy.

To test out a practical use of these control parameters to create new sounds they experiment with sound interpolation. Taking inspiration from [27] to hybridize two different musical sounds, and comparing how PCA, LSTM-AE and VAEs behave. They selected pairs of sounds, created separate encodings for each sound in the pair, and did an entry-wise linear interpolation of the two latent vectors. This interpolation was then decoded and reconstructed using the inverse STFT and Griffin and Lim algorithm. The results showed LSTM-AE outperforming VAE which outperformed PCA, LSTM-AE managed to preserve the note attacks, PCA had blurrier harmonics and audio artifacts.

Based on the review of related work, it can be concluded that autoencoders are great for dimensionality reduction, feature extraction and reconstruction purposes, (even denoising and anomaly detection as seen in [21]) but lack certain properties to make them good at generating new unseen data. This is due to the deterministic way they map their input on to the latent space. The input is mapped onto a fixed and isolated latent variable creating a discontinuous latent space. [30] This is done with no regularization leading to an asymmetric latent space not centred around an origin, and to different labels to have different sized clusters. Sampling the latent space for a random unseen latent variable will lead to undesirable output as the model has no input on what to generate. [29]

3.4 Variational Autoencoders (VAE)

Variational autoencoders represent a significant advancement in the domain of generative models. By combining the power of Variational Bayesian methods with the architecture of autoencoders, they are capable of efficiently generating high quality samples from complex data distributions. While traditional autoencoders deterministically map x into a latent vector z of fixed coordinates in the latent space. VAEs learn to map x as a multivariate probability distribution in the latent space

parameterised by the mean and variance of each variable in the latent space. This effectively creates a continuous and structured latent space, allowing the VAEs to generate new data by sampling from these distributions.

To understand this, we can think of the dataset D as consisting of N independent and identically distributed samples of some variable x , each conditioned on an unobserved random variable z sampled from a distribution $p_\theta(z)$. This relationship between x and z can be expressed through the joint distribution $p_\theta(x, z) = p_\theta(x|z)p_\theta(z)$. Extending this framework to a directed graphical model with latent variables allows us to represent the joint distribution $p_\theta(x, z)$ and compute the marginal likelihood of our observable data w.r.t parameters θ , $p_\theta(x)$ by integrating over z : [31]

$$p_\theta(x) = \int p_\theta(x, z) dz \quad (6)$$

However, this integration over z does not have an analytic solution or an efficient estimator, making it intractable. This intractability is related to the intractability of the true posterior density given by $p_\theta(z|x) = \frac{p_\theta(x, z)}{p_\theta(x)}$. If the joint probability $p_\theta(x, z)$ was tractable, a tractable posterior would lead to a tractable marginal likelihood and vice versa, but unfortunately both are intractable in Deep Latent Variational Models. [33]

Instead of directly computing the true posterior we can approximate it using a parametric inference model $q_\varphi(z|x)$. This model is called the probabilistic encoder, and we optimize its parameters φ to approximate to the true posterior:

$$q_\varphi(z|x) \approx p_\theta(z|x) \quad (7)$$

It's called probabilistic because given a datapoint x it returns a probabilistic distribution over the possible values of z from which the datapoint x could have been generated.[31] The encoder outputs two vectors defining the parameters (mean μ , and variance σ^2) of the probability distributions over the latent variables, these distributions are most often assumed to be gaussian. A latent vector z is then sampled from these distributions.

$$(\mu, \log \sigma^2) = \text{EncoderNeuralNet}_\varphi(x) \quad (8)$$

$$q_\varphi(z|x) = \mathcal{N}(z; \mu, \text{diag}(\sigma^2)) \quad (9)$$

The decoder network $p_\theta(x|z)$ reconstructs the input data from that latent vector z passed from the encoder, or it can be used to generate new unseen data by discarding the encoder and passing a random sample from a standard distribution through the decoder. [39]

3.4.1 The ELBO

The optimization objective of the variational autoencoder, is the evidence lower bound (ELBO).

This objective can be derived for any inference model $q_\varphi(z|x)$ with variational parameters φ , the following way:

$$\log p_\theta(x) = \mathbb{E}_{q_\varphi(z|x)}[\log p_\theta(x)] \quad (10)$$

$$= \mathbb{E}_{q_\varphi(z|x)} \left[\log \left[\frac{p_\theta(x, z)}{p_\theta(z|x)} \right] \right] \quad (11)$$

$$= \mathbb{E}_{q_\varphi(z|x)} \left[\log \left[\frac{p_\theta(x, z) q_\varphi(z|x)}{p_\theta(z|x) q_\varphi(z|x)} \right] \right] \quad (12)$$

$$= \mathbb{E}_{q_\varphi(z|x)} \left[\log \left[\frac{p_\theta(x, z)}{q_\varphi(z|x)} \right] \right] + \mathbb{E}_{q_\varphi(z|x)} \left[\log \left[\frac{q_\varphi(z|x)}{p_\theta(z|x)} \right] \right] \quad (13)$$

$$= \mathbb{E}_{q_\varphi(z|x)} [\log p_\theta(x, z) - \log q_\varphi(z|x)] + D_{KL}(q_\varphi(z|x) \parallel p_\theta(z|x)) \quad (14)$$

[33]

The term on the right-hand side is the Kullback-Leibler divergence (D_{KL}) between the approximate posterior $q_\varphi(z|x)$ and the true posterior $p_\theta(z|x)$. The D_{KL} is always positive or equal to zero when the two probability distributions are identical. The term on the left-hand side is the ELBO also called the variational lower bound ($\mathcal{L}_{\theta, \varphi}(x)$). Switching things around:

$$\mathcal{L}_{\theta, \varphi}(x) = \log p_\theta(x) - D_{KL}(q_\varphi(z|x) \parallel p_\theta(z|x)) \quad (15)$$

Due to the D_{KL} being non-negative:

$$\mathcal{L}_{\theta, \varphi}(x) \leq \log p_\theta(x) \quad (16)$$

The D_{KL} between the approximate and the true posteriors represents the gap between the ELBO and the marginal likelihood, this is also called the tightness of the bound. By maximizing the ELBO with respects to the θ and φ parameters we maximize the marginal likelihood $p_\theta(x)$ and minimize the distance between the approximate and the true posteriors. [33]

We can also write the ELBO as:

$$\log p_\theta(x) \geq \mathcal{L}_{\theta, \varphi}(x) = \mathbb{E}_{q_\varphi(z|x)} [\log p_\theta(x|z)] - D_{KL}(q_\varphi(z|x) \parallel p_\theta(z)) \quad (17)[31]$$

Stochastic gradient descent (SGD) is used to optimize the lower bound with respects to both variational parameters θ and φ . SGD is an optimization algorithm used to minimize a given loss function by iteratively adjusting the model parameters. Unlike Batch Gradient descent that adjusts the model parameters based on the gradient computed over the entire dataset for each iteration, SGD updates its parameters based on an estimate of the gradient that is computed from a randomly selected single datapoint or small subset of the data (also known as minibatch). [35]

It is intractable to calculate the gradient for an individual-datapoint ELBO, however there are good

unbiased estimators that use minibatch SGD to obtain the gradients with respects to θ . This can be boiled down to a simple Monte Carlo estimator. But we run into some problems when it comes to the gradients with respects to the φ parameter. We cannot use a simple Monte Carlo estimator due to z being a random variable sampled from the approximate posterior $q_\varphi(z|x)$ this leads to very high variance in the gradient estimation w.r.t φ . [31] We run into another problem when using the SGD algorithm, which is that its not possible to directly backpropagate the gradients through the random variable z . [33] This is because the sampling process introduces a non-continuous operation that has no gradient breaking the chain of gradient computation required for backpropagation. [39]

3.4.2 Reparameterization trick

The solution to this problem is the *reparameterization trick*, which can be seen in figure 2. The reparameterization trick works by getting the randomness out of the gradient estimation process. Instead of sampling z directly from the approximate posterior distribution $q_\varphi(z|x)$, we define z as a deterministic function expressed as $z = g(\varphi, x, \epsilon)$. By introducing a random noise variable ϵ sampled from a distribution $\epsilon \sim p(\epsilon)$ that is independent of φ and x we are then able to backpropagate through z and compute the gradients of ELBO with respects to φ .

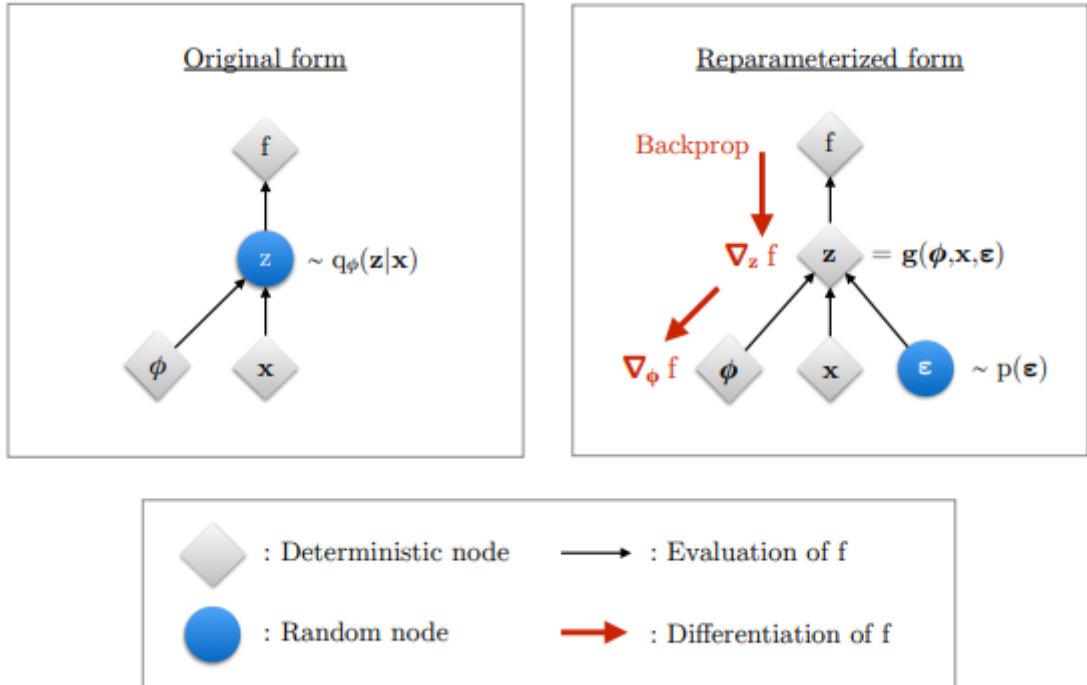


Figure 2 Taken from [33]

With this reparameterization we can define the ELBO optimization as:

$$\epsilon \sim p(\epsilon) \tag{18}$$

$$z = g(\varphi, x, \epsilon) \quad (19)$$

$$\tilde{\mathcal{L}}_{\theta,\varphi}(x) = \log p_{\theta}(x, z) - \log q_{\varphi}(z|x) \quad (20)$$

A common choice for ELBO optimization is a simple factorized gaussian encoder, which defines the noise variable ϵ as being sampled from a univariate gaussian distribution and choosing an invertible function for the $g(\cdot)$ transformation that will make it easy to compute. This is defined as:

$$\epsilon \sim \mathcal{N}(0, \mathbf{I}) \quad (21)$$

$$(\boldsymbol{\mu}, \log \boldsymbol{\sigma}) = \text{EncoderNeuralNet}_{\varphi}(x) \quad (22)$$

$$\mathbf{z} = \boldsymbol{\mu} + \boldsymbol{\sigma} \odot \boldsymbol{\epsilon} \quad (23)$$

Where $\odot$ denotes an element wise multiplication, $\mathbf{I}$ denotes an identity matrix. The transformation function g now defines z as the sampled noise vector, scaled by a diagonal covariance matrix $\boldsymbol{\sigma}$ and shifted by the mean $\boldsymbol{\mu}$ vector.

As mentioned above ELBO can be expressed as:

$$\mathcal{L}_{\theta,\varphi}(x) = \mathbb{E}_{q_{\varphi}(z|x)} [\log p_{\theta}(x|z)] - D_{KL}(q_{\varphi}(z|x) || p_{\theta}(z)) \quad (24)$$

Using the reparameterization trick for gradient-based optimization we can get a training objective for a single data point x^i :

$$p_{\theta}(z) = \mathcal{N}(z; 0, \mathbf{I}) \quad (25)$$

$$\mathcal{L}_{\theta,\varphi}(x^i) \approx \frac{1}{2} \sum_{j=1}^J \left(1 + \log \left((\sigma_j^i)^2 \right) - (\mu_j^i)^2 - (\sigma_j^i)^2 \right) + \frac{1}{L} \sum_{l=1}^L \log p_{\theta}(x^i | z^{(i,l)}) \quad (26)$$

Where $\mathbf{z}^{(i,l)} = \boldsymbol{\mu}^i + \boldsymbol{\sigma}^i \odot \boldsymbol{\epsilon}^l$ and $\boldsymbol{\epsilon}^l \sim \mathcal{N}(0, \mathbf{I})$ [31]

The first term is a regularization term that was derived from the Kullback-Leibler divergence between the approximate posterior $q_{\varphi}(z|x)$ and the chosen prior distribution $p_{\theta}(z)$. The chosen prior distribution is typically an isotropic multivariate gaussian distribution centered at zero. This regularization encourages the approximate posterior distribution to approach the chosen prior distribution, thereby normalizing and smoothing the latent space. A smooth and continuous latent space allows for more consistent and interpretable sampling, and better interpolation between latent variables. But at a possible cost of some overlapping between variables leading to a noisier encodings and potential loss of distinct latent representations. [36]

The second term is the expected log-likelihood of the observed data given the latent vector z ; it measures how well the decoder network is expected to reconstruct the input data x given z . This is calculated using a Monte Carlo estimator. This means that for every datapoint x we sample z many times and we average the log likelihood over all samples taken. This metric neglects the shape of the latent space. Having this term by itself would mean the network would behave like an autoen-

coder. As the model would train the variance of the latent variables would decrease and collapse to a single point, leading to a discontinuous latent space. Although it would improve the peak reconstruction quality it would be very hard to consistently generate good data from random samples. [36]

3.4.3 The β -VAE

In [38] Higgins et al introduced the β -VAE to optimize the ELBO for different tasks, its intended use being to encourage disentanglement between latent variables. Disentanglement means that each latent variable captures one specific interpretable generative factor of the data. These factors should be independent from one another. Meaning that changing one latent variable should only affect the corresponding factor in the data and not influence any other factor captured by other latent variables. To do this, they introduced the β variable to the ELBO equation, allowing them to adjust the constraint on the structure of the latent space imposed by the KL divergence term.

The modified ELBO is given by:

$$\mathcal{L}_{\theta,\varphi}(x) = \mathbb{E}_{q_{\varphi}(z|x)} [\log p_{\theta}(x|z)] - \beta D_{KL}(q_{\varphi}(z|x)||p_{\theta}(z)) \quad (27)$$

Depending on the value assigned to β , one can choose to prioritize disentanglement with $\beta > 1$, prioritize reconstruction with $\beta < 1$, or keep it as a standard VAE with $\beta = 1$. Higher β usually means higher disentanglement of the latent representation but may induce a larger variance loss for reconstructed data, leading to blurrier images. Lowering β improves the quality of reconstructed images, but as mentioned previously this would make it approach the behaviour of an autoencoder, losing continuity of the latent space and leading to worse generation of data from random sampling of the latent space and worse interpolation. It is important to find a balance as if it is not properly tuned the regularization term can lead to what is known as over-pruning, meaning the network only considers a limited number of latent variables for generation.[36]

3.4.4 RAVE: A variational autoencoder for fast high-quality neural audio synthesis [34]:

In this study Caillon et al introduced RAVE, a variational autoencoder designed for real-time high-quality audio synthesis. They proposed a two-part training model using a variational autoencoder with a β parameter for representation learning, setting the base of the model, finetuning the model with a Generative Adversarial Network (GAN) based approach.

The ELBO of the β -VAE was modified to use multiscale spectral distance instead of reconstruction loss, this is an amplitude spectrum-based distance metric used to avoid penalising the model for inaccurate phase reconstruction, focusing on more perceptually relevant features of the signal.

After training the VAE to learn a satisfactory representation of the data, they freeze the encoder and train the decoder as the generator of the GAN component to finetune the model. This encourages

the decoder to produce signals that are as close as possible to the real data. In the adversarial loss function they also include the spectral distance penalty and a feature matching loss proposed by Kumar et al in [41]. This feature matching loss intends to minimize the L1 distance between the discriminator feature maps of real and generated signals within the feature map produced by the discriminator. The objective function to finetune the decoder is the following:

$$\mathcal{L}_{total}(x, z) = \mathcal{L}_{gen}(z) + \mathbb{E}_{\hat{x} \sim p(x|z)}[S(x, \hat{x}) + \mathcal{L}_{FM}(x, \hat{x})] \quad (28)$$

Post-training they introduce a method to identify and retain the most informative parts of the latent space. This is done by creating a matrix of the latent space with the uninformative parts zeroed out and applying Singular Value Decomposition (SVD) on it. The fidelity parameter f is then used to control the trade-off between the reconstruction fidelity and the compactness of the latent model. This approach saves them the trouble of determining the latent space dimensionality before training, allowing them to estimate it and adjust the learned representation accordingly post training.

They manage to achieve very efficient synthesis of 48kHz audio signals due to multiband decomposition of the raw waveform. The multiband decomposition allows them to decrease temporal dimensionality and therefore expand the temporal receptive field without a major performance impact.

RAVE is capable of modelling both monophonic and polyphonic audio signals with different styles and configurations, having been trained on the “Strings” and the “VCTK” datasets, the former being composed of 30 hours of raw audio sampled at 48kHz of stringed instruments with both monophonic and polyphonic configurations, with solo and group performances and different styles; the later being composed of 44 hours of raw audio also sampled at 48kHz of speech of 110 different speakers with several different accents. They trained two other state of the art models that utilize autoregressive generation called “NSynth” and “SING” on a versions of these datasets that were down-sampled to 16kHz. These models were used for evaluation purposes on unsupervised modelling.

To evaluate their method on synthesis quality, they had 33 participants in 15 trials comparing the quality of audio samples from either the original ground truth data, or from the 3 models. Although there was a large gap between RAVE and the ground truth, it did have the best rated samples of the three models. They explain that the large gap might come from the difficulty in capturing the different room acoustics present in the original dataset.

RAVE managed to achieve audio synthesis twenty times faster than real-time on a CPU and up to 240 times faster on a GPU. The multiband decomposition speeds up the model by a factor of 25. RAVE is significantly faster than NSynth and SING. They tested timber transfer by reconstructing violin samples on a model trained on speech, getting great results.

3.5 Vector-Quantized Variational Autoencoder (VQ-VAE)

In [42] Van Den Oord et al introduced the Vector Quantized Variational Autoencoder (VQ-VAE) model. This model innovates on the VAE framework by incorporating vector quantization to learn a discrete latent space and a learned prior distribution. They argue that a discrete latent representation is more of a natural fit to certain modalities such as Language which is typically described as a sequence of symbols, Images which can be described with concise language, complex reasoning and predictive learning. This model is effective at addressing the problem of “posterior collapse” found in many VAE architectures with powerful decoders. The “posterior collapse” is the problem of decoders essentially ignoring the latent variables z when reconstructing x .

The VQ-VAE architecture consists of an encoder, an embedding dictionary and a decoder. The embedding dictionary is composed of K embedding vectors, each having the same shape as the encoder output. After encoding x into z the model finds the embedding vector e_k that most resembles z , creating a categorical posterior distribution $q(z|x)$ with its probabilities defined as follows:

$$q(z = k|x) = \begin{cases} 1 & \text{for } k = \operatorname{argmin}_j \|z_e(x) - e_j\|_2 \\ 0 & \text{otherwise} \end{cases} \quad (29)$$

Where $\operatorname{argmin}_j \|z_e(x) - e_j\|_2$ finds the index j that gives the minimum Euclidean distance between the output of the encoder $z_e(x)$ and encoding vector e_j . The embedding vector e_k is then inputted into the decoder as $z_q(x)$.

$$z_q(x) = e_k, \text{ where } k = \operatorname{argmin}_j \|z_e(x) - e_j\|_2 \quad (30)$$

The decoder generates an image from the embedding vector. During back-propagation the gradient is copied from the decoder input $z_q(x)$ to the encoder output $z_e(x)$ as there is not real way to get a real gradient from the mapping function.

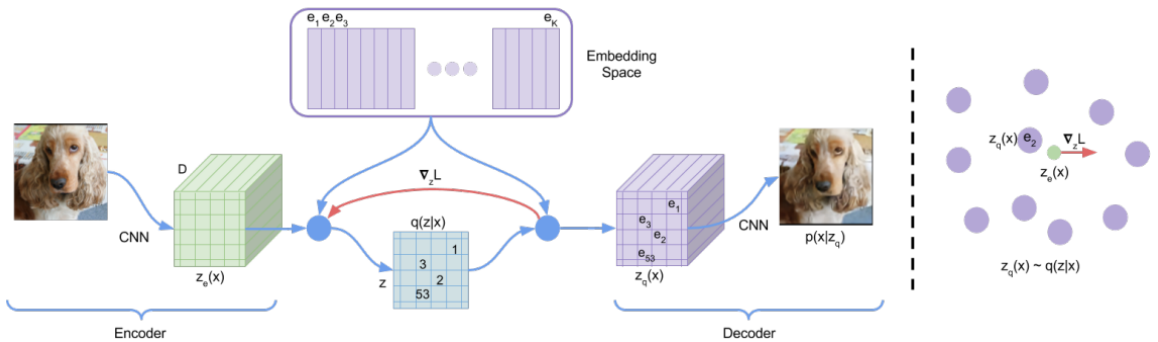


Figure 3: From [41], VQ-VAE architecture on the left, Embedding space visualization of the right.

In figure 3 on the right, we can see a visualization of the embedding space. The gradient changes the output of the encoder $z_e(x)$ to minimize reconstruction loss, which might change the embedding vector that is closest to it in the next forward pass.

The overall loss function is given by:

$$L = \log p(x|z_q(x)) + \|sg[z_e(x)] - e\|_2^2 + \beta \|z_e(x) - sg[e]\|_2^2 \quad (31)$$

The loss function is composed of three terms. The first term is the reconstruction error which optimizes the encoder and decoder by minimizing the difference between the original data x and its reconstruction. This term does not have any effect on the embeddings due to the “straight-through” gradient estimation that copied the gradient from the decoder to the encoder.

To optimize the embedding space the two other terms are introduced. The second term is the Vector Quantization objective, which uses l_2 norm to bring the embedding vectors closer to the encoder outputs $z_e(x)$. This uses the “stop-gradient” (sg) operator on $z_e(x)$ to not have any effect on the encoder outputs and solely impact the embedding vector dictionary. This sg function acts as an identity operator in the forward pass and has zero partial derivatives that restrict its operand to be a non-updated constant. The third term is called the commitment loss, it penalizes the encoder for growing too far from the nearest embedding vector. It uses l_2 norm to encourage the encoder to “commit” to the nearest embedding vector and a β parameter to control the strength of this penalty. Here the sg operator is on the embedding vector as to only optimize the encoder.

In contrast to VAEs where the prior distribution is static, in VQ-VAEs the prior is learned. Here, the prior distribution is categorical, meaning that the latent variables z are discrete and take on values from a finite set. This prior distribution can be made autoregressive, allowing the distribution of a latent variable to depend on the values of other latent variables in the latent space. During training, the prior distribution is assumed to be uniform, therefore the KL term of the ELBO is constant with respect to the encoder parameters and can be ignored. After training the VQ-VAE, the prior is then adjusted to better fit these learnt discrete latent variables, using an autoregressive model such as PixelCNN for images and WaveNet for raw audio.

After fitting the prior, the VQ-VAE can generate new data by sampling from the learned discrete latent space. The VQ-VAE was shown to be capable of capturing very long term dependencies. This was demonstrated by its ability to generate 128x128 colour images, by successfully generating video sequences conditioned on actions such as turning right or moving forward without any degradation to visual quality whilst keeping the local geometry accurate, and by being able to generate meaningful chunks of speech using an unconditional model, capable of voice transfer. Their experiments showed that they manage to capture key features of the data in an unsupervised fashion. Whilst achieving likelihoods almost on par with their continuous latent variable counterparts.

3.5.1 VQ-VAE 2 [43]:

In [43] Razavi et al expanded on VQ-VAE model, introducing VQ-VAE 2. The major differences being a multi-scale hierarchical architecture, a more powerful autoregressive model for prior training, the use of exponential moving averages (EMA) to update the dictionary introduced in [42] as

an alternative to the codebook loss and the introduction of classifier-based rejection sampling.

The new architecture consists of a hierarchy of latent spaces. The encoder first down-samples the original data to a smaller representation, which is quantized to the bottom-level latent space. Another set of residual blocks further compresses the data to an even smaller representation, quantizing it into the top-level latent space. The bottom latent space is conditioned on the top latent. This design encourages the encoding of complementary information and avoids redundancy in the latent representations. The top latent space captures the global information such as shape and geometry of objects while more localized information such as texture and small details are captured by the bottom latent. Different models can be used to train the prior over the different latent spaces allowing for bespoke models capable of capturing correlations specific to each level.

The priors are trained using deep autoregressive models from the state-of-the-art PixelCNN family, after training the VQ-VAE model. To better capture the global structural information, the prior over the top latent map is equipped with multi-headed self-attention layers, allowing it to cover distant spatial correlations. The bottom latent map encodes local information at a higher resolution, using large conditioning stacks rather than self-attention, due to memory constraints. The priors can be trained separately, allowing them to train larger models.

The EMA is an algorithm normally used in algorithms such as K-means to find the optimal centroids for clustering. Used in this context to replace the codebook loss for a faster and more stable update to the codebook vectors [44] [5]. As it cannot be directly used to update the dictionary when working with minibatches, van den Oord defines an online version of it as such:

$$N_i^t := N_i^{(t-1)} * \gamma + n_i^t(1 - \gamma) \quad (32)$$

$$m_i^t := m_i^{t-1} * \gamma + \sum_j z_{i,j}^t(1 - \gamma) \quad (33)$$

$$e_i^t := \frac{m_i^t}{N_i^t} \quad (34)$$

Where n_i^t is the number of encoder output vectors $z_e(x)$ in the mini batch that are closest to the codebook item e_i and γ is a decay parameter with a value between 0 and 1. The value used in both [42] and [43] being $\gamma = 0.99$.

A classifier-based rejection sampling method is introduced to trade-off diversity and quality of samples. This method utilizes a pre-trained classifier to score the samples, according to the probability the classifier assigns to the correct class.

The results showed that VQ-VAE 2 is a simple light-weight method capable of generating diverse high-resolution images capable of rivalling state-of-the-art GANs in terms of sample quality, fidelity and diversity. The Negative log likelihood (NLL) results also demonstrated effective generalization without overfitting.

3.5.2 Open AI's Jukebox [5]:

In 2020, Dhariwal et al introduced Jukebox, an approach to model and generate high-fidelity music that with singing, in the raw audio domain. Jukebox utilizes a multi-scale hierarchical VQ-VAE to compress the input audio into a latent embedding space, which is then modelled using autoregressive transformers. The model can be conditioned on several aspects, such as artist, lyrics, midi, genre, timing etc. allowing to steer generation in unique ways.

Its architecture consists of a multi-scale VQ-VAE model with three levels of abstraction, the top level being the prior, and the lower two being up-samplers. The model can be conditioned on lyrics and midi by swapping out the top prior. To model the raw audio, Wavenet style 1-D convolutions are used. Higher levels have larger hop-lengths than lower levels. A larger hop-length corresponds to a more compressed and broader representation capturing longer temporal dependencies.

When using VQ-VAE 2 for raw audio they noticed the top level is under-utilized sometimes even experiencing complete collapse, to maximize the information stored at each level they decided to train separate autoencoders for each level with varying hop-lengths. The output of the model at each level is independent to the other levels.

To avoid codebook collapse which is when all encodings get mapped into a single or a few embedding vectors, they randomly reset the codebook vectors to one of the encoder outputs when it falls below average usage. This ensures all codebook vectors are being used.

In addition to the usual loss functions found in VQ-VAEs they include the spectral loss. This is due to the failure to capture mid-to-high frequencies using only the sample-level reconstruction loss. This allows the model to match the spectral components whilst ignoring the phase component.

After training the separate VQ-VAEs a simplified version of a transformer with sparse attention autoregressive model which they call Scalable Transformer is used to model the prior and up-samplers over the compressed space. The up-sampler transformers are conditioned on the upper-level codes. At each level, the transformers model longer temporal dependencies whilst maintaining the same computational footprint, this is due to the increase in hop-length in the upper levels.

Conditioning the model with genre and artist not only allows for better customization in music generation but also reduces entropy in audio prediction which enables the model to achieve better quality within the particular styles. They attach a timing signal which includes information on the song length and the position of the sample within the song, this allows the model to capture patterns within song structures.

Lyric conditioning proved to be a very complex task especially when compared to the regular Text-to-speech task. The challenge comes from needing to model the temporal alignment of lyrics and singing, the singer's voice, the plethora of ways a certain lyric can be sang depending on the song's context, from a conditioning signal that only contains the text of the lyrics.

They use an encoder-decoder style model to condition on lyrics, where the encoder is an auto-

regressive transformer tasked with capturing relevant features of the lyrics, that the music decoder is conditioned on to generate music tokens. This model is trained on 24 second chunks of audio. They use tools such as Spleeter to isolate the vocal tracks in the chunks and NUS AutoLyricsAlign on the isolated vocals to produce word-level alignments of the lyrics.

After training the VQ-VAEs, up-samplers and top-level priors, new songs are generated by sampling from these priors and up-samplers, from top to bottom. The sampling is controlled by conditioning information such as genre, artist and lyrics. The lower-level prior models are also conditioned on the codes from the upper levels. Codes for each level are generated one token at a time through a process called ancestral sampling, where in the generation of each token is conditioned on every previously generated token. The VQ-VAE decoder then converts the bottom-level codes into audio. To generate longer music, they use windowed sampling, where they move the sampling window ahead by half the context length and continue sampling conditioned on this half context. They can create new song completions, by using primed sampling. This is done by forward passing a real audio segment into the model to generate the corresponding prior codes, which are then used as the initial tokens in the ancestral sampling process, allowing the model to fill in the rest of the song.

To train Jukebox they gathered a dataset consisting of 1.2 million songs, paired with lyrics and metadata. They trained the model using 44.1kHz raw audio, applying data augmentation through random downmixing the right and left channels to mono. They used 1 billion parameters for the up-samplers and 5 billion parameters for the top-level prior. This significantly improved fidelity and allowed for a better modelling of the broader distribution of songs, resulting in samples with better musicality, longer coherence and initial singing, at the expense of longer rendering time. To get intelligible singing they limited the dataset to songs primarily in English and trained a seq2seq model with lyric conditioning.

Jukebox showed great capacity in generating samples that maintain long coherence and being able to structure the songs with beginning, middle, and end. Though it fails to capture long term musical patterns such as repeating melodies and chorus, due to the short context length of the top-level prior. The model captures familiar harmonies fits the singing in a very natural sounding way, with consideration for the rhythm and melody of the overall song and style. Though it fails to match the creativity and memorability found in human compositions, lacking interesting melodic patterns such as the antecedent and consequent pattern familiar to many melodies. It also fails to properly align the lyrics for certain genres such as hip hop and rap. The model is able to generate a variety of distinct but still relevant samples from a fixed set of conditions, showing great diversity within niche created by the applied conditions. Though deviations from these conditions were noted. The model struggled to generalize certain artists to unrelated music genres, but was capable of interpolating artist embeddings to create new recognizable voices. It manages quite well to adapt to new lyrics. It is quite useful for generating different continuations of incomplete songs.

Overall Jukebox is a very impressive model for generating music pieces multiple minutes long, but it is very computationally demanding. Having taken multiple weeks to train the model, the top level prior and the up-samplers. It also takes very long to generate a full minute sample, taking about one hour to generate the top-level tokens and about 8 hours to up-sample it which makes it quite impractical as a tool.

3.6 Generative Adversarial Networks (GANs)

Generative Adversarial Networks have become one of the cornerstones in the field of generative machine learning models. They were developed by Ian Goodfellow and his colleagues in 2014 in [45]. It consists of two multilayer perceptrons: the Generator and the Discriminator.

The Generator model takes in a low dimensional random noise vector sample from a fixed noise distribution and outputs a high dimensional data sample. The Discriminator judges samples from both the real dataset and from the Generator, scoring each sample on their probability of belonging to the original data distribution.

Having studied VAEs I like to think of the Generator as a Decoder that takes a latent vector from a fixed noisy latent space and decodes it to match the dataset's distribution. Improving its weights with guidance from the Discriminator's scores, rather than a reconstruction loss. The latent space, usually a multivariate gaussian distribution, remains fixed as there is no encoder.

These two models play a two-player minmax game with the value function $V(D, G)$:

$$\min_G \max_D V(D, G) = \mathbb{E}_{x \sim p_{data}(x)} [\log D(x)] + \mathbb{E}_{z \sim p_z(z)} [\log (1 - D(G(z)))] \quad (35)$$

This is the traditional GAN objective function. Where the first term is the expected log probability the discriminator gives to a real data sample for being part of the data distribution and the second term is the expected log probability the discriminator gives a generated sample for not being part of the data distribution, hence the $1 - D(G(z))$. The Discriminator seeks to maximize this function while the Generator seeks to minimize it. The training process involves alternating between optimizing the Discriminator and the Generator to outsmart each other. Whilst the idea is quite straightforward, training a GAN can be quite unstable and complex. To understand where this instability comes from let's get into the theoretical nitty gritty.

Given a fixed generator, we can derive the shape of the optimal Discriminator:

$$D^*(x) = \frac{P_{data}(x)}{P_{data}(x) + P_g(x)} \quad (36)$$

3.6.1 Jensen-Shannon Divergence

The optimal discriminator is able to completely distinguish real data and generated data. The loss function of this optimal discriminator is: (these derivations are shown in [45])

$$L(D^*, G_\theta) = 2 JSD(P_{data} \parallel P_g) - 2 \log 2 \quad (37)$$

Where JSD is the Jensen-Shannon divergence between the real data probability distribution and the generated data probability distribution. Minimizing the objective function as a function of θ results in minimizing this Jensen-Shannon Divergence when the discriminator is optimal. One would think that the best course of action would be to alternate between training the discriminator as close as possible to optimality and then to do gradient steps on θ . However, in practice, as the discriminator improves the generator becomes harder to optimize. [46]

If we train a discriminator until optimality its error will go to 0, and the JSD will be maxed out. This suggests that the distributions (real and generated) are either not continuous or have disjointed supports.

The support of a distribution is the set of all possible values where the probability density function is non-zero. In other words, for a random variable X with a distribution P_X , its support is the set of values x , where $P_X(x) > 0$. Disjointed supports mean that the supports of these two distributions have no overlap between them. Two distributions are said to be continuous if there are no gaps or isolated points in the data where the probability suddenly drops to zero between their supports.

A possible explanation for this discontinuity is that the supports of the distributions lie on low dimensional manifolds, meaning their data distributions are concentrated on small lower-dimensional subsets of the entire higher-dimensional space. These low dimensional manifolds have practically no chance of perfectly aligning as any small perturbation can cause misalignment. This implies that there always exists a smooth optimal discriminator between the two distributions that has perfect accuracy. The JSD and even D_{KL} will also be maxed out even if the two manifolds lie close to each other and the generated samples look incredibly similar to real data. When these divergencies are maxed out it is impossible to minimize them by gradient descent. [46] This suggests that a measure that accounts for distance between points in the manifolds should be used.

As the discriminator is improved the cost function gets closer to JSD, if it is almost impossible for it to not be maxed out, it means that the gradient of the generator will vanish. This leaves the training as a sort of balancing act with how well to train the discriminator. Train it too well it won't provide useful feedback to the generator, train it too little the updates to the generator will be inaccurate due to poor discrimination. [46]

3.6.2 Instability in GANs

Goodfellow et al in [45] propose the use of an alternative cost function for the generator when dealing with a discriminator with high confidence, to be used particularly in early training when the generator outputs poor results. Instead of training the generator to minimize $\log(1 - D(G(z)))$, they propose training it to maximize $\log D(G(z))$. While this function does not suffer from vanishing gradients it can cause massive instability in updates and mode dropping. This is due to the different form the cost function will take when the discriminator reaches optimality. For an optimal discriminator fixed for a value θ_0 :

$$\mathbb{E}_{z \sim p(z)}[-\nabla_{\theta} \log D^*(G_{\theta}(z))|_{\theta=\theta_0}] = \nabla_{\theta}[D_{KL}(P_{g_{\theta}} \parallel P_{data}) - 2JSD(P_{g_{\theta}} \parallel P_{data})|_{\theta=\theta_0}] \quad (38)$$

Where the D_{KL} is not the maximum likelihood as the distributions are swapped and the negative JSD is pushing the two distributions away instead of bringing them closer. KL divergence is known to cause mode dropping. Mode dropping means that the generator fails to cover the full spectrum of the real data distribution, leading to a lack of diversity in generated samples and a lack of representation of certain features of the real data distribution. [47]

In [46] they show that the gradient updates output from this alternative cost function follow a Cauchy distribution. Cauchy distributions have infinite variance and expectation, which means the updates will be highly unstable.

To address these problems, several variants of GANs have been introduced, some focus on changing the objective function and its associated distance metric to stabilize training and improve convergence; others focus on constraining the Discriminator through gradient penalties and normalization to offset the unbounded loss functions and stop the gradients from vanishing; while others change the model architectures. [49]

3.6.3 Wasserstein GAN

In [48] Arjovsky et al introduced the Wasserstein GAN (WGAN) which proposes a new objective function that under an optimal discriminator (called a critic in this case) equates to minimizing the Earth's Movers Distance (also known as the Wasserstein distance) between the real P_{data} and the generated distributions P_g :

$$W(P_{data}, P_g) = \sup_{\|f\|_L \leq 1} \mathbb{E}_{x \sim P_{data}(x)}[f(x)] - \mathbb{E}_{x \sim P_g}[f(x)] \quad (39)$$

Where $\|f\|_L \leq 1$ denotes that the function f is 1-Lipschitz.

This metric incorporates the notion of distance between the elements within the support of the real distribution and the ones in the support of the generated distribution, inducing a notion of distance between the manifolds. [46] Instead of the discriminator being trained to identify samples as real or

fake, which equates to minimizing the problematic JSD they instead train it on a function that assists computing the Wasserstein distance. [3]

This new value function looks like:

$$V_{WGAN}(D_w, G) = \mathbb{E}_{x \sim P_{data}(x)}[D_w(x)] - \mathbb{E}_{z \sim P_z(z)}[D_w(G(z))] \quad (40)$$

To ensure D_w remains a 1-Lipschitz function, Arjovsky in [48] proposes the use of weight clipping.

3.6.4 WaveGAN [3]:

In [3] Donahue et al introduced the WaveGAN and the SpecGAN, the first generative adversarial networks applied to audio synthesis. WaveGAN taking a raw audio approach, whilst SpecGAN took on spectrogram approach. They both use a model architecture based on the Deep Convolutional GAN (DCGAN) introduced in [50] by Radford and Metz.

DCGAN was a major contributor to the overall success of GANs in the unsupervised image synthesis domain. It was one of the first models to manage to successfully scale up GAN using CNNs. They introduced a few architectural innovations such as replacing pool layers with strided convolutions, implementing batch normalization, eliminating fully connected hidden layers for deeper architectures, and using activation functions such as ReLU and LeakyReLU. These changes helped stabilize the training process, allowing to train with higher resolutions and deeper generative models. [50]

To train on raw waveforms in WaveGAN they flattened the DCGAN architecture to take in the one-dimensional input. They noted that natural audio signals are more likely to exhibit periodicity and correlations across large windows than natural images. With that in mind they widened the receptive field of the transposed convolutions of the generator, replacing the two-dimensional filters of size 5x5 with longer one-dimensional filter of length 25 and up-sample each layer by a factor of 4 instead of the usual 2. For the discriminator they also use the same widened one-dimensional filters and increase the stride. This leaves the WaveGAN with about the same number of parameters as DCGAN, and after adding one more layer results in an output of 16384 audio samples, being about one second of audio at 16kHz. This length is enough for the audio domains they are exploring.

For all experiments they decided to use WGAN-GP algorithm. Introduced by Gulrajani et al in [51], it improved on the WGAN by replacing the weight clipping approach with a gradient penalty to enforce the Lipschitz constraint. They use this algorithm as it showed significant capabilities in training a variety of model configurations where other GAN losses have failed. Which proved to be useful as it produced reasonable results. [3]

The upsampling done by the transposed convolutions often leave artifact frequencies, which the discriminator can easily detect as they always occur at a particular phase. This leads to an overly

confident discriminator, which might inhibit optimization. They propose the phase shuffle operation to randomly perturb the phase of each of the Discriminator's layer's activations by adding or subtracting a number of samples within the range $[-n, n]$ before the next layer's input. This will encourage the discriminator to be invariant to the phase of the input waveform.

SpecGAN trains on a spectrogram representation of the audio, this allows it to keep the two-dimensional form of the original DCGAN architecture. To process the audio into this format they use the short-time Fourier transform (STFT). To be able to directly compare the two models (WaveGAN and SpecGAN) they also use the same dimensionality per unit of time, yielding a 128x128 spectrogram image. The magnitude of the resultant spectra is scaled in amplitude to better align with human perception. The model is trained to generate new spectrogram images which can then be approximately converted into audio through a lossy process called the Griffin-Lim algorithm. This process attempts to estimate the phase of these newly generated sounds, though it is far from a perfect estimator, leaving the audio with poor sound quality.

They trained both models on several datasets but most notably the Speech commands Zero through Nine (SC09). This dataset is composed of recordings of individual spoken digits with many different speakers and recording conditions. This dataset is particularly useful as it allows for easier human assessment of the quality of the generated samples. Whilst the other datasets such as Drum sound effects, bird vocalizations and piano most likely would require experts to analyse how close they resemble their real counterparts.

They used Inception score and nearest neighbour comparisons to evaluate both models. The Inception score was measured using an audio classifier model trained on SC09. This classifier transforms the audio input into Mel-spectrograms and classifies each audio into the 10 classes of SC09. This approach seems to be biased towards models trained with spectrograms due to their shared representation. They use two nearest neighbour indicators to ensure the Inception score is not unfairly giving high scores when the results don't match it.

They use human judgment to evaluate the generated audio qualitatively. Asking the annotators to label which digit they hear and to assign subjective values to the sound quality, intelligibility and speaker diversity.

The results didn't show a clear winner between the two models. The qualitative evaluation showed that the WaveGAN performed better in sound quality and speaker diversity according to the annotator's preference. But the SpecGAN performed better in accuracy, with the annotators being able to correctly guess more digits in comparison to the WaveGAN generated audio, this could mean SpecGAN is better at capturing the variance in the underlying data. The poor sound quality from the SpecGAN samples is most likely due to the lossy Griffin-Lim algorithm.

Overall, both models showed great potential in the field of unsupervised audio generation. Wave-

GAN is fully parallel allowing it to produce an hour of audio in less than 2 seconds. WaveGAN showed good performance in other datasets, being able to capture semantic modes for drum sounds, generate distinct bird vocalizations, producing musically consonant piano motifs and creating speech like babbling with the TIMIT dataset.

3.6.5 Parallel WaveGAN [52]:

In [52] Yamamoto et al proposed Parallel WaveGAN (no relation to the previously mentioned WaveGAN from [3]). This is a lightweight supervised waveform generation approach using the GAN framework. The Generator is a WaveNet based model, with some key differences such as the use of non-causal convolutions, its input being random noise drawn from a Gaussian distribution (typical of GANs), and it not being an autoregressive model. The Generator is conditioned on log-mel spectrograms. To train the model they used the Least Square GAN (LSGAN) objective function proposed by Mao et al in [53], along with their proposed Multi-resolution STFT auxiliary loss, for the generator's loss function.

$$L_G(G, D) = L_{aux}(G) + \lambda_{adv}L_{adv}(G, D) \quad (41)$$

Where λ_{adv} is an hyperparameter to balance the loss terms.

The Least square GAN utilizes a different objective function that essentially minimizes a distance metric called the Pearson X^2 divergence between P_{data} and P_g . Proposed as a better alternative to the original GAN's objective function.

$$L_{adv}(G, D) = \mathbb{E}_{z \sim P_z(z)} \left[\left(1 - D(G(z)) \right)^2 \right] \quad (42)$$

The proposed Multi-resolution auxiliary loss works by taking multiple STFTs of the generated audio with different analysis parameters such as FFT size, window size, etc. They take the STFT loss of each STFT component, which is composed of a spectral convergence loss and a log magnitude loss, then average the loss across all resolutions. This loss term helps the generator learn the time-frequency characteristics of speech whilst preventing it from becoming overfit to a fixed STFT representation.

Human evaluation showed that it was almost on par with other Vocoder models such as ClariNet in terms of perceptual quality, but it was able to generate speech signals 1.96 times faster and train its model 4.82 times faster than ClariNet. Parallel WaveGAN showed great potential, achieving competitive results to contemporary Transformer-based Text-to-Speech models, whilst generating 24 kHz speech waveforms 28.68 times faster than real-time with a relatively light weight model.

3.6.6 MelGAN [41]:

In [41] Kumar et al introduced MelGAN, a lightweight GAN model for Mel-spectrogram inversion and conditional waveform synthesis. It was the first GAN model capable of high-quality raw audio generation without additional distillation or perceptual loss functions.

Unlike regular GANs that take in a gaussian noise vector as input, the Generator of MelGAN takes in Mel-spectrograms and up-samples them through stacks of transposed convolutional layers and residual blocks with dilated convolutions, into the raw waveform output. These residual blocks with dilations are added to each upsampling layer to induce long-range correlations. The receptive field at each stack of dilated convolutions increases exponentially, leading to a larger overlap between distant time-steps. There is little perceptual difference in results when feeding additional noise to the generator as the conditioning information is very strong.

Careful selection of kernel-size, stride and dilations prevent the checkerboard artifacts mentioned above in the WaveGAN discussion. This is a simple alternative to the phase suffling approach. Weight normalization was used in all layers of the generator and discriminator models for better training dynamics, as unlike other normalization techniques, it does not limit the capacity of the discriminator or normalize the activations.

They use a multi-scale architecture with three Discriminators with identical structures but operating at different audio resolutions. This approach allows the discriminators to focus on different frequency ranges, leading to a more complete discriminator. These discriminators are Markovian window-based discriminators which learn to classify between a distribution of small audio chunks rather than a full audio sequence. These windows have a large overlap which helps it to maintain coherence across the chunks. The use of grouped convolutions allows for the use of larger kernel sizes whilst maintaining a small number of parameters. Leading to a very efficient fast lightweight approach that can capture essential high frequency structures at variables lengths of audio.

For the objective function they use an alternative to the original GAN objective called the Hinge Loss GAN objective, given by:

$$\min_{D_k} \mathbb{E}_x[\min(0, 1 - D_k(x))] + \mathbb{E}_{s,z}[\min(0, 1 + D_k(G(s, z)))] , \forall k = 1,2,3 \quad (43)$$

$$\min_G \mathbb{E}_{s,z} \left[\sum_{k=1,2,3} -D_k(G(s, z)) \right] \quad (44)$$

Where x is the raw waveform, s is the conditioning information (mel-spectrograms) and z is the gaussian noise vector. This objective is used in models such as BigGAN [49] achieving state-of-the-art results.

In addition to this adversarial hinge loss, they use a feature matching objective to train the genera-

tor. This minimizes the L1 distance between discriminator feature maps of real and synthetic audio. This acts as a learned similarity metric, where the discriminator learns to differentiate fake from real data outside the raw audio space. They do not use any additional noise as mentioned above as this hurts the sound quality.

.MelGAN has demonstrated to be significantly faster it's contemporary alternatives achieving real-time performance without hardware-specific optimizations. MelGAN was shown to be a robust model capable of being plugged into different architectures and efficiently carrying out different tasks, such as music translation and unconditional music synthesis. It substituted the wavenet autoregressive decoder of the Universal Music Translation Network by Mor et al [54]. This is an autoencoder model used to change the style, instrumentation and genres of music. After adjusting the MelGAN decoder hyperparameters to obtain the right input resolution, and training for 4 days in each selected music domain, the model was able to achieve music translation with decent quality at 2500 times the speed of the original model.

They also substitute a high-capacity autoregressive WaveNet decoder of a VQ-VAE used for unconditional music generation, following the structure proposed by van den Oord et al in [42].

Various experimentation such as speech synthesis, music domain translation and unconditional music generation showed that it is comparable in quality to state-of-the-art models whilst being quick, lightweight and robust.

3.7 Diffusion Models

3.7.1 Introduction

Diffusion models have recently blown up in popularity with the advent of stable diffusion, DALL E, Midjourney, etc. It has been proven quite successful at generating high quality art and hyper realistic imagery.

The diffusion probabilistic model was first proposed by Sohl-Dickstein et al in 2015 [16], but only rose to prominence around 2020, driven by the impressive results showcased in [17]. Showing it could generate high quality samples on par if not better than other state of the art generative models such as GANs.

A diffusion model is a parameterized Markov chain using variational inference to produce samples matching the data after finite time. [17] A parameterized Markov chain is a random process that undergoes transition from one state to another within a state space, adhering to the Markov property. The Markov property states that the probability of transitioning to any particular state de-

depends solely on the current state and the time elapsed and not on the sequence of states that preceded it. The fact that its parameterized means that some parameters can be tuned to adjust the behaviour of the Markov chain. These parameters are learned using variational inference, this is an optimization algorithm to approximate an inferred complex distribution to the true complex distribution.

The diffusion model consists of two processes, a forward process also known as the diffusion process, that takes an image x_0 and gradually adds small amounts of Gaussian noise to it according to a noise schedule β_t , until the original signal is destroyed x_T ; and a reverse process that gradually undoes the diffusion process, turning a noisy image back into its original form. This reverse process is done by training a model to restore the structure of the data, leading to a highly flexible and tractable generative model of the data. [16]

3.7.2 Denoising Diffusion Probabilistic Models (DDPMs)

Like Variational autoencoders, Diffusion models are a category of latent variable models, but of a different form: $p_\theta(x_0) := \int p_\theta(x_{0:T})dx_{1:T}$, where $x_1, \dots, x_T$ are latent variables of the same dimensionality as the data $x_0 \sim q(x_0)$. [17] The approximate posterior of this latent variable model is the forward process, unlike variational autoencoders this is not parameterized by a neural network, it is instead fixed to a Markov chain that progressively adds Gaussian noise to the data according to a linear variance schedule $\beta_{1:T}$:

$$q(x_{1:T}|x_0) = \prod_{t=1}^T(q(x_t|x_{t-1})), \quad q(x_t|x_{t-1}) = \mathcal{N}(x_t; \sqrt{1 - \beta_t}x_{t-1}, \beta_t \mathbf{I}) \quad (45,46)$$

To allow for quick sampling of x_t at any arbitrary timestep t given the original image x_0 , without needing to iterate through every single timestep between 0 and t , the following notation is used:

$$\alpha_t = 1 - \beta_t \quad \bar{\alpha}_t = \prod_{s=1}^t \alpha_s \quad q(x_t|x_0) = \mathcal{N}(x_t; \sqrt{\bar{\alpha}_t}x_0, (1 - \bar{\alpha}_t)\mathbf{I}) \quad (47,48,49)$$

The reverse process is defined as a Markov chain with learned gaussian transitions starting at a standard normal distribution $p(x_T) = \mathcal{N}(x_T, 0, \mathbf{I})$:

$$p_\theta(x_{0:T}) = p(x_T) \prod_{t=1}^T p(x_{t-1}|x_t) \quad p_\theta(x_{t-1}|x_t) = \mathcal{N}(x_{t-1}; \mu_\theta(x_t, t); \Sigma_\theta(x_t, t)), \quad (50,51)$$

While VAEs have two models to parameterize the mean μ_θ and the variance Σ_θ of the output distribution x_{t-1} , here the variance is fixed to the variance schedule $\beta_{1:T}$, so only one model is needed which is used to predict the mean of the distribution, or as I'll discuss later to predict the noise added between two timesteps. β_t is kept small as this ensures the forward and reverse processes have identical functional forms. [16]

It is intractable to directly estimate the negative log loss function $-\log p_\theta(x_0)$, due to the complexity of integrating over all possible latent variables and noise processes involved in the

generation of the observed data. Instead, akin to VAEs, a variational bound is used. [56] Similar to VAEs where the divergence between the true and the approximate posteriors is subtracted to the log likelihood, this bound is given by adding the divergence between the forward and the reverse process to the negative log loss function:

$$\mathbb{E}[-\log p_\theta(x_0)] \leq \mathbb{E}[-\log p_\theta(x_0)] + D_{KL}(q(x_{1:T}|x_0) \parallel p_\theta(x_{1:T}|x_0)) \quad (52)$$

This can then be rewritten to:

$$\mathbb{E}[-\log p_\theta(x_0)] \leq \mathbb{E}_q \left[-\log p \left(\frac{p_\theta(x_{0:T})}{q(x_{1:T}|x_0)} \right) \right] = \mathbb{E}_q \left[-\log p(x_T) - \sum_{t \geq 1} \log \frac{p_\theta(x_{t-1}|x_t)}{q(x_t|x_{t-1})} \right] \quad (53)$$

And rewritten further until we get:

$$L = \mathbb{E}_q \left[\underbrace{D_{KL}(q(x_T|x_0) \parallel p(x_T))}_{L_T} + \sum_{t \geq 1} \underbrace{D_{KL}(q(x_{t-1}|x_t, x_0) \parallel p_\theta(x_{t-1}|x_t))}_{L_{t-1}} - \underbrace{\log p_\theta(x_0|x_1)}_{L_0} \right] \quad (54)$$

Where L_T called sometimes the prior matching term, measures how close the image in its final step is to a standard Gaussian prior.[56] This has no trainable parameters and is constant during training, therefore it can be ignored. L_{t-1} called the denoising matching term, measures the KL divergence between the predicted noise transition $p_\theta(x_{t-1}|x_t)$ and a sort of ground truth $q(x_{t-1}|x_t, x_0)$ which defines how to denoise a noisy image x_t given the original image x_0 . [56] Note that this form of the forward process is an inversion of the diffusion process and not the same as $q(x_t|x_{t-1})$, this new term appears by applying the bayes rule on the forward process:

$$q(x_{t-1}|x_t, x_0) = \frac{q(x_t|x_{t-1}, x_0)q(x_{t-1}|x_0)}{q(x_t|x_0)} \quad (55)$$

This conditioning on x_0 makes this forward process posterior tractable as not knowing the original image the model is trying to denoise would lead to enormous variance in the process. These two loss terms are comparisons between two Gaussian distributions, and therefore can be expressed in closed form instead of high variance Monte Carlo estimates. L_0 is the reconstruction term, it measures how close the output image of the denoising transition between x_1 and x_0 is to the original image.

With the variance being fixed to the β_t schedule we can adjust this L_{t-1} divergence to focus solely on the divergence between means of the predicted and ground truth distributions, as follows

$$q(x_{t-1}|x_t, x_0) = \mathcal{N}(x_{t-1}; \tilde{\mu}_t(x_t, x_0), \tilde{\beta}_t \mathbf{I}), \quad (56)$$

$$\text{Where } \tilde{\mu}_t(x_t, x_0) = \frac{\sqrt{\bar{\alpha}_{t-1}}\beta_t}{1-\bar{\alpha}_t}x_0 + \frac{\sqrt{\bar{\alpha}_t(1-\bar{\alpha}_{t-1})}}{1-\bar{\alpha}_t}x_t \text{ and } \tilde{\beta}_t = \frac{1-\bar{\alpha}_{t-1}}{1-\bar{\alpha}_t}\beta_t \quad (57,58)$$

$$p_\theta(x_{t-1}|x_t) = \mathcal{N}(x_{t-1}; \mu_\theta(x_t, t); \Sigma_\theta(x_t, t)) \quad (59)$$

$$L_{t-1} = \mathbb{E}_q \left[\frac{1}{2\sigma^2} \|\tilde{\mu}_t(x_t, x_0) - \mu_\theta(x_t, t)\|^2 \right] + C \quad (60)$$

This is a weighted mean square error between the means of the distributions.

Using the reparameterization trick also used in VAEs $\mathcal{N}(\mu, \sigma^2) = \mu + \sigma \cdot \epsilon$, we can get:

$$x_t(x_0, \epsilon) = \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1-\bar{\alpha}_t}\epsilon \quad (61)$$

and writing it in terms of x_0 :

$$x_0(x_t, \epsilon) = \frac{1}{\sqrt{\bar{\alpha}_t}}(x_t - \sqrt{1-\bar{\alpha}_t}\epsilon) \quad (63)$$

where ϵ is noise sampled from a standard normal distribution $\epsilon \sim \mathcal{N}(0, \mathbf{I})$. Subbing it into L_{t-1} we get:

$$L_{t-1} - C = \mathbb{E}_{x_0, \epsilon} \left[\frac{1}{2\sigma^2} \left\| \tilde{\mu}_t(x_t(x_0, \epsilon), \frac{1}{\sqrt{\bar{\alpha}_t}}(x_t(x_0, \epsilon) - \sqrt{1-\bar{\alpha}_t}\epsilon) - \mu_\theta(x_t(x_0, \epsilon), t) \right\|^2 \right] \quad (63)$$

$$= \mathbb{E}_{x_0, \epsilon} \left[\frac{1}{2\sigma^2} \left\| \frac{1}{\sqrt{\bar{\alpha}_t}} \left(x_t(x_0, \epsilon) - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}}\epsilon \right) - \mu_\theta(x_t(x_0, \epsilon), t) \right\|^2 \right] \quad (64)$$

This equation shows us that μ_θ must predict $\frac{1}{\sqrt{\bar{\alpha}_t}}(x_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}}\epsilon)$ given x_t . With x_t being a input to the model it can be parameterized like:

$$\mu_\theta(x_t, t) = \tilde{\mu}_t \left(x_t, \frac{1}{\sqrt{\bar{\alpha}_t}} \left(x_t - \sqrt{1-\bar{\alpha}_t}\epsilon_\theta(x_t) \right) \right) = \frac{1}{\sqrt{\bar{\alpha}_t}} \left(x_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}}\epsilon_\theta(x_t, t) \right) \quad (65)$$

With $\epsilon_\theta(x_t, t)$ being a model to predict the noise ϵ from x_t . x_{t-1} can be sampled by computing $x_{t-1} = \frac{1}{\sqrt{\bar{\alpha}_t}}(x_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}}\epsilon_\theta(x_t, t) + \sigma_t z$, where $z \sim \mathcal{N}(0, \mathbf{I})$. This parameterization can be put into the loss function to further simplify it to:[17]

$$\mathbb{E}_{x_0, \epsilon} \left[\frac{\beta_t^2}{2\sigma_t^2 \alpha_t (1-\bar{\alpha}_t)} \left\| \epsilon - \epsilon_\theta(\sqrt{\bar{\alpha}_t}x_0 + \sqrt{1-\bar{\alpha}_t}\epsilon, t) \right\|^2 \right] \quad (66)$$

$$\mathbb{E}_{x_0, \epsilon} \left[\frac{\beta_t^2}{2\sigma_t^2 \alpha_t (1 - \bar{\alpha}_t)} \|\epsilon - \epsilon_\theta(x_t, t)\|^2 \right] \quad (67)$$

In conclusion, this L_{t-1} loss term can be used to train the reverse process to predict the mean $\tilde{\mu}_t$ or we can modify it to predict the noise ϵ .

To train L_0 Ho et al [17] first scales the image data to a linear range between $[-1, 1]$ to ensure the reverse process works with consistently scaled inputs that align with the standard normal distribution. They then introduce an independent discrete decoder based on the following gaussian distribution: $\mathcal{N}(x_0; \mu_\theta(x_1, 1), \sigma_1^2 \mathbf{I})$ to compute discrete log-likelihoods. This method ensures that the variational lower bound is a lossless codelength of discrete data, without the need for additive noise or incorporating the Jacobian of the scaling operation in the log likelihood.

This method is ultimately not used in their best performing model, finding that a simpler loss function leads to better sample quality.

$$L_{simple}(\theta) = \mathbb{E}_{t, x_0, \epsilon} [\|\epsilon - \epsilon_\theta(x_t, t)\|^2] \quad (68)$$

This variant of the variational bound includes a case for the reverse process from $x_1 \rightarrow x_0$ where instead of using the discrete decoder method to compute L_0 , they instead treat it as any other timestep while not adding any noise to the data in its computation. The weight imposed by the L_{t-1} loss function on the divergence between the noise variables was also removed, this leads to down-weight loss terms at smaller t timesteps. Which is beneficial as these down-weight loss terms denoise the data with very small amount of noise, making the network focus on harder denoising tasks in larger t timesteps.

Algorithm 1 Training	Algorithm 2 Sampling
1: repeat 2: $\mathbf{x}_0 \sim q(\mathbf{x}_0)$ 3: $t \sim \text{Uniform}(\{1, \dots, T\})$ 4: $\epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ 5: Take gradient descent step on $\quad \nabla_\theta \ \epsilon - \epsilon_\theta(\sqrt{\alpha_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon, t)\ ^2$ 6: until converged	1: $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ 2: for $t = T, \dots, 1$ do 3: $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ if $t > 1$, else $\mathbf{z} = \mathbf{0}$ 4: $\mathbf{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(\mathbf{x}_t - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_\theta(\mathbf{x}_t, t) \right) + \sigma_t \mathbf{z}$ 5: end for 6: return $\mathbf{x}_0$

Figure 4 Taken from [17]

These are the algorithms used for training and sampling, using the simplified loss function.

In experiments they used 1000 timesteps t , a variance schedule β_t increasing linearly from $\beta_1 = 0.0001$ to $\beta_T = 0.02$. The model uses a U-Net based architecture for the reverse process.

The U-Net architecture was introduced in 2015 by Ronneberger et al [58] as a convolutional network for image segmentation tasks in the biomedical field but has since gained popularity in other

tasks and fields, becoming the most popular choice of architecture for generative models in particular diffusion models. It uses an encoder-decoder structure where the input is downsampled through convolutional and pooling layers to capture high-level features, and then upsampled through convolutions and up-convolutional layers to reconstruct the input. It has skip connections that directly connect layers in the encoder to their corresponding layers in the decoder. These connections help alleviate the vanishing gradient problem and help preserve features from the encoder part of the model, allowing the decoder to use this information to generate more accurate and detailed outputs. [59,60]

This U-net architecture is based on an unmasked PixelCNN++, having replaced the weight normalization throughout the model with group normalization, for a simpler implementation. The diffusion timestep is given to each residual block by the Transformer sinusoidal position embedding. The same parameters are used across all timesteps. They used 4 feature maps for the smaller resolution models and 6 for the larger ones, having two convolutional residual blocks per resolution level and self-attention at the 16x16 resolution between convolutional blocks.

They used Inception, FID and negative log likelihood scores to evaluate the sample quality having achieved better FID using their unconditional model than most concurrent models. Training revealed that using the true variational bound L_{t-1} achieves better codelengths meaning its better in terms of data compression, whilst the simplified objective L_{simple} results in better sample quality. They found that predicting the mean $\tilde{\mu}$ only worked on the true variational bound, whilst predicting the noise variable ϵ worked as well using the variational bound and even better in the simplified unweighted objective. They also found that learning the reverse process variances instead of using the fixed variance schedule led to unstable training and worse sample quality.

The lossless codelengths produced by the models trained on CIFAR10, whilst better than most energy-based models they are far behind other types of likelihood-based generative models. Coupled with their high-quality samples it can be concluded that DDPMs are great lossy compressors.

Interpolation in the latent space between two images can be achieved by encoding the images with the forward process, essentially adding noise to the images, interpolating between the latents, then using the reverse process on the mixed image to denoise it. Different t timesteps can be chosen for differing levels of reconstruction, with the larger timesteps producing more varied interpolations that match the original images less and less.

3.7.3 Improved DDPM

In [57] Open Ai's Nichol et al, introduced the Improved Denoising Diffusion Probabilistic model, with the aim of addressing a lot of the problems found with the original DDPM [17]. The major improvements being the implementation of a learnable variance given by an interpolation between

β_t and $\tilde{\beta}_t$; a more informative noise schedule; a hybrid learning objective that takes into account the new learned variance as well as the simplified learning objective of [17] improving the overall log-likelihood of the model; using 4000 timesteps instead of the usual 1000 for training; and using as few as 50 timesteps for sampling.

While the DDPM by Ho et al was able to achieve great sample quality as reflected by its FID and sample scores, the log-likelihoods of the models were not competitive with concurrent likelihood-based generative models. A small improvement in log-likelihood can have a drastic impact on sample quality and learned feature representation [17]. Changing the timesteps to 4000 already improved the log-likelihood from 3.99 bits/dim to 3.77.

In [17] they noticed that fixing the variance σ_t^2 to either β_t or $\tilde{\beta}_t$ made no difference in sample quality. Nichol et al figured out by plotting the ratio $\tilde{\beta}_t/\beta_t$ over the diffusion timesteps, that whilst the two β s represent opposite extremes they result in very similar variance. Being almost identical with only a small window of timesteps near $t = 0$ where they diverge from each-other. They also found that the more steps added to the diffusion process the smaller that window became. This suggested that as more steps were added the choice of σ_t made little difference to sample quality, meaning that most impact came from the model's mean $\mu_\theta(x_t, t)$. Though the same cannot be said for the log-likelihood, as good choice of $\Sigma_\theta(x_t, t)$ can potentially improve it.

They decided to learn $\Sigma_\theta(x_t, t)$ as an interpolation between β_t and $\tilde{\beta}_t$ in the log domain:

$$\Sigma_\theta(x_t, t) = \exp(v \log \beta_t + (1 - v) \log \tilde{\beta}_t) \quad (69)$$

Where v is a vector containing one component per dimension. Despite v being unconstrained, allowing the network to predict variances beyond the interpolated range, the network did not do that, suggesting that the bounds are expressive enough.

They also found the linear noise schedule used in [17] to be sub-optimal as the end of the forward process is too noisy to contribute much to the sample quality. They designed a more informative schedule using the $\cos^2$ function, that spreads out the extremes, changing them very slowly to prevent abrupt changes in noise level, and keeps the middle with a linear drop-off. This allows for a more gradual and informative noise process in the extremes.

To involve the learned variance into the objective function they create a hybrid objective:

$$L_{\text{hybrid}} = L_{\text{simple}} + \lambda L_{\text{vib}} \quad (70)$$

With λ (set to 0.001) being a parameter used to balance the losses so L_{vib} does not overwhelm L_{simple} .

$$L_{\text{vib}} = \mathbb{E}_{t \sim p_t} \left[\frac{L_t}{p_t} \right], \text{ where } p_t \propto \sqrt{\mathbb{E}[L_t^2]} \text{ and } \sum p_t = 1 \quad (71)$$

L_{vlb} is quite unstable to optimize directly due to a uniform sampling of t causing unnecessary noise in the objective. To address this, they use importance sampling, maintaining a history of the previous 10 values for each loss term, dynamically updating $\mathbb{E}[L_t^2]$ accordingly. This allows for direct optimization of the L_{vlb} objective, leading to their best log-likelihood results.

Experimentation showed that using the hybrid objective along with the cosine schedule improves the log-likelihood whilst keeping the sample quality the same as the original DDPM. Further optimizing L_{vlb} leads to a better log-likelihood but at the cost of sample quality.

With this improved version of the DDPM they were able to produce high quality samples with much fewer timesteps in sampling, making it possible. They are able to reduce the sampling timesteps by selecting a subsequence evenly distributed timesteps from the full set used during training, adjusting the noise schedule and variances according to the new subsequence. The model was able to achieve near optimal sample quality with just 100 timesteps.

3.7.4 Denoising Diffusion Implicit Models (DDIM)

In [55] Song et al present Denoising Diffusion Implicit models, which presents a non-Markovian alternative to the forward process, proving to be a much more efficient and flexible alternative to the traditional DDPM sampling methods. Allowing for sampling with much fewer timesteps without sacrificing much sample quality. These non-Markovian diffusion processes lead to the same DDPM training objective, meaning there is no need to train a separate U-net model to use this sampling method. Reframing the sampling process as an implicit model, introduces a deterministic component to an otherwise stochastic process. This leads to much more meaningful encodings of samples enabling much smoother and coherent interpolations within the latent space.

As the traditional DDPM objective L_{simple} in [17] only depends on the marginal distribution $q(x_t|x_0)$ rather than the full joint distribution $q(x_{1:T}|x_0)$. They explored using a non-Markovian process to reach one that directly depend on that joint distribution. In this paper they encapsulate the DDPM objective into L_γ defined as:

$$L_\gamma(\epsilon_\theta) = \sum_{t=1}^T \gamma_t \mathbb{E}_{x_0 \sim q(x_0), \epsilon_t \sim N(0,I)} \left[\left\| \epsilon_\theta^t (\sqrt{\alpha_t} x_0 + \sqrt{1 - \alpha_t} \epsilon_t) - \epsilon_t \right\|_2^2 \right] \quad (72)$$

Where $\gamma_{1:T}$ is a vector of positive coefficients dependent on $\alpha_{1:T}$. This objective accommodates for several types of models including DDPMs (by setting γ as a vector of 1s), noise conditional score networks, and also supports non-Markovian diffusion processes. If the parameters θ of the models ϵ_θ^t are not shared across different timesteps then the optimal solution will not depend on the weights of γ , this means that the optimal solution in one model such as DDPMs is also the optimal solution for any other model that can be represented by L_γ such as the non-Markovian generative

process they present. Therefore, we can use the L_1 objective as a surrogate training objective to train our Unet model on and use the non-markovian forward and reverse process for sampling the latent space.

The inference distribution ultimately used was:

$$q_\sigma(x_{1:T}|x_0) = q_\sigma(x_T|x_0) \prod_{t=2}^T q_\sigma(x_{t-1}|x_t, x_0) \quad (73)$$

Where $q_\sigma(x_T|x_0) = \mathcal{N}(\sqrt{\alpha_T}x_0, (1 - \alpha_T)\mathbf{I})$, this term indicates that the final term is directly influenced by the original state, which means it will retain meaningful information about the original state, where in DDPMs by the time you get to the final timestep it contains little to no trace of the original state.

For all $t > 1$ they use:

$$q_\sigma(x_{t-1}|x_t, x_0) = \mathcal{N}\left(\sqrt{\alpha_{t-1}}x_0 + \sqrt{1 - \alpha_{t-1} - \sigma_t^2} * \frac{x_t - \sqrt{\alpha_t}x_0}{\sqrt{1 - \alpha_t}}, \sigma_t^2 \mathbf{I}\right) \quad (74)$$

With the mean function design to ensure the marginal distribution retains the desired form: $q_\sigma(x_t|x_0) = \mathcal{N}(\sqrt{\alpha_t}x_0, (1 - \alpha_t)\mathbf{I})$ for all t . From here they can define the forward process using the bayes rule:

$$q_\sigma(x_t|x_{t-1}, x_0) = \frac{q_\sigma(x_{t-1}|x_t, x_0)q_\sigma(x_t|x_0)}{q_\sigma(x_{t-1}|x_0)} \quad (75)$$

This is a non-Markovian process as the current state (x_t) not only depends on the previous state (x_{t-1}) as well as the original state (x_0). Here σ controls the stochasticity of the forward process, setting σ to 0 the process becomes deterministic, meaning all randomness is removed and we can determine with absolute certainty the full trajectory between x_0 and x_t given both those states

With this forward process in mind now they need a reverse process leveraging both the current and the initial states. Using Eq.

(

$$x_0(x_t, \epsilon) = \frac{1}{\sqrt{\alpha_t}}(x_t - \sqrt{1 - \alpha_t}\epsilon) \quad (63)$$

) they can predict the initial state given x_t and use it to predict the next state:

$$p_\theta^t(x_{t-1}|x_t) = \begin{cases} \mathcal{N}(x_0^{(1)}(x_1, \epsilon_\theta^{(1)}), \sigma_1^2 \mathbf{I}) & \text{if } t = 1 \\ q_\sigma(x_{t-1}|x_t, x_0(x_t, \epsilon_\theta^{(t)})) & \text{otherwise} \end{cases} \quad (76)$$

Training using DDIMs is done by optimizing the variational inference objective:

$$J_\sigma(\epsilon_\theta) = \mathbb{E}_{x_{0:T} \sim q_\sigma(x_{0:T})} [\log q_\sigma(x_{1:T}|x_0) - \log p_\theta(x_{0:T})] \quad (77)$$

This objective function fits within scope of the L_γ , with every possible σ being able to be represented by a certain γ weighting such that $J_\sigma = L_\gamma + C$. Therefore, we only need to train one model for every choice of σ .

Sampling is done as stated previously from a pretrained DDPM model, with the non-Markovian reverse and forward processes. Using Eq.(76) we can generate x_{t-1} given x_t by computing if $t > 1$:

$$x_{t-1} = \sqrt{\alpha_{t-1}} \left(\frac{x_t - \sqrt{1 - \alpha_t} \epsilon_\theta^{(t)}(x_t)}{\sqrt{\alpha_t}} \right) + \sqrt{1 - \alpha_{t-1} - \sigma_t^2} * \epsilon_\theta^{(t)}(x_t) + \sigma_t \epsilon_t \quad (77)$$

Where $\epsilon_t \sim \mathcal{N}(0, I)$, here we are using the parameterization trick Eq.(23) on Eq.(74) while using Eq. (

$$x_0(x_t, \epsilon) = \frac{1}{\sqrt{\alpha_t}} (x_t - \sqrt{1 - \alpha_t} \epsilon) \quad (63)$$

) for to predict x_0 in the first term in of the equation. Different generative processes can be achieved by choosing different σ , for example the DDPM generative process results from setting

$$\sigma_t = \sqrt{\frac{1 - \alpha_{t-1}}{1 - \alpha_t}} * \sqrt{\frac{1 - \alpha_t}{\alpha_{t-1}}}, \quad (78)$$

for all t . By setting σ_t to 0 for all t , we get a deterministic forward process given x_{t-1} and x_0 , except when $t=1$, in Eq.(77) the coefficient on the third term becomes 0, meaning no additional noise is added to the generation of x_{t-1} . This deterministic generative model is an implicit probabilistic model where samples are generated from latent variables with a fixed procedure. This leads to a more meaningful and structured latent space, where given some latent variable x_T you can consistently generate the same output x_0 .

The generalization of the L_1 objective to other forward processes allows to consider forward processes with fewer timesteps, as long as $q_\sigma(x_t|x_0)$ is fixed. The authors define a new sequential forward process over a subset $x_{\tau_1:\tau_S}$ of the original diffusion process $x_{1:T}$, where τ is an increasing subsequence of the original forward process trajectory. This results in a generative process that samples latent variables according to the reverse sequence of τ referred to as the sampling trajectory. This approach significantly enhances computational efficiency, enabling for accelerated sampling while maintaining sample quality, all without the need for retraining the model.

By rewriting the DDIM iterative equation the authors revealed its similarity to the Euler integration for solving ordinary differential equations (ODEs), suggesting that the sampling trajectory can be interpreted as a discrete approximation to a continuous process. By discretizing this ODE finely enough it is possible reverse the generation process. This reversibility allows the model to simulate the forward and reverse trajectories of the ODE, providing a structured way to move between the

image space and the latent space in a continuous and deterministic manner. With this well-structured latent space, it becomes possible to interpolate within it, to generate smooth and semantically meaningful transitions between different data samples.

Experiments were performed on 4 image different datasets of various resolutions, on a UNet model with the roughly the same hyperparameters as in [17] to facilitate direct comparison. Sample quality was measured with FID. Training was performed using the L_1 objective with $T = 1000$ with no changes from the original DDPM training procedure. Sampling was done using their DDIM objective, adjusting two parameters: σ controlling the stochasticity and S controlling how many steps τ were used in the sampling trajectories. To simplify comparisons, they write σ in the following form:

$$\sigma_{\tau_i}(\eta) = \eta \sqrt{\frac{1 - \alpha_{\tau_{i-1}}}{1 - \alpha_{\tau_i}}} \sqrt{\frac{1 - \alpha_{\tau_i}}{\alpha_{\tau_{i-1}}}} \quad (79)$$

Where $\eta \in \mathbb{R}_{\geq 0}$ allows for a simpler transition between DDPM ($\eta = 1$) and DDIM ($\eta = 0$) processes. They also show results for η in between these extremes and past the DDPM extreme denoted as $\hat{\sigma}$ where the random noise has a larger standard deviation than the DDPM. They also showed results for various steps S smaller than T .

Results show that DDIMs significantly outperformed DDPMs in terms of sample quality with S up to 20 times smaller than T . Enabling speed ups of 10 to 100 times faster than the original DDPM sampling without sacrificing sample quality.

The determinism of DDIMs means that the latent space can retain high-level features regardless of the generation trajectory. Experimentation showed this clearly as when starting from an initial latent variable x_T the DDIMs generated images with similar high-level features across different trajectories. This makes them particularly useful for tasks such as reconstruction and interpolation, which cannot be said for DDPMs due to their stochastic nature.

Interpolating between two latent variables using DDIMs revealed great results with semantically meaningful, smooth and coherent morphs between two samples.

3.7.5 DiffWave:

3.7.6 WaveGrad:

3.7.7 Noise2Music[7]:

3.8 Evaluation Methodology

Evaluation of generative models is quite challenging and complex. Unlike classification models whose performance can be measure on accuracy against a ground truth, the success of a generative model is much harder to quantify and often requires multiple metrics. These evaluations sometimes require human intervention or even the use of models trained to assess quality, some aligning with human assessment, while others are tailored to specific objective criteria.

This difficulty in evaluation is due to the overall objective generative models, which is to derive the unknown probability distribution $p_r(x)$ which the real data x is drawn from. The model encodes a distribution $p_g(x)$ over new generated samples with the aim of finding a generative distribution that is as close as possible to the real data distribution. [61]

Some models are capable of directly evaluating $p_g(x)$, when this is the case the most common option is to calculate the likelihood of the test dataset under p_g . However, a good likelihood does not always result in good sample quality, and vice versa. Moreover, some latent variable models such as GANs lack an explicit representation of the probability distribution p_g , making alternative evaluation metrics useful if not vital. The most commonly used metrics to evaluate generative models include:

3.8.1 Log-Likelihood

As previously stated, where available the log likelihood is the de-facto standard for evaluating and even training generative models. Although it can be computationally intractable to measure for many models. Some models requiring indirect metrics such as the variational bounds seen in diffusion models and VAEs, and approximations using the Jensen-Shannon divergence or Wasserstein distance, or the many other distance metrics used to optimize GANs, to evaluate and optimize the model.

The practical application of log-likelihood in evaluation generative models poses many problems, particularly in high-dimensional data. The discrete nature of image data often requires preprocessing steps such as dequantization, where uniform noise is added to transform the discrete pixel values into continuous data. This process ensure that the log-likelihood remains meaningful, avoiding artificially inflated values due to the discrete distribution's differential entropy being negative infinity. [62]

While log-likelihood is a crucial metric for density estimation of the underlying data distribution it does not always correlate with the visual quality of generated samples. This discrepancy becomes evident when experimenting with different divergences to approximate the data distribution, leading to varying log-likelihood results.

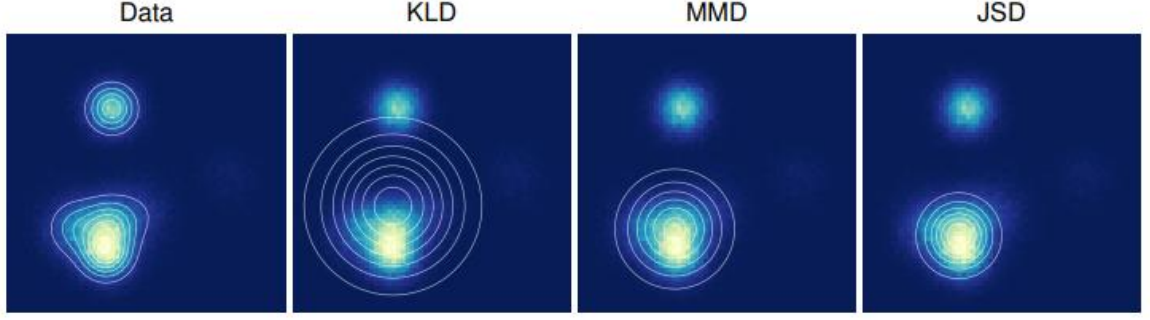


Figure 5: Taken from [62]: Comparison of different divergence measures when fitting a Gaussian distribution to a mixture of Gaussians.

Fig.(5) shows how different divergence measures affect the log-likelihood and sample quality. Using the KL Divergence metric, the probability mass is spread across the data space resulting in high average log-likelihood but assigning a lot of probability mass to non-data regions. In contrast MMD and JSD focus on one of the data modes producing more plausible data but potentially having a lower log-likelihood. Whilst the data extracted from JSD model will be much more accurate to the data distribution compared to KLD model, it fails to capture the whole data distribution, meaning the resulting samples, while visually pleasing, may not reflect the full diversity of the dataset, limiting the model's robustness.

Ultimately, while log-likelihood may not correlate with the visual fidelity of generated samples, it remains a crucial metric for specific applications such as inpainting, denoising or classification, where the overall fit to the data distribution is more important than sample plausibility. This emphasises how important it is to not rely solely on one metric and to understand the trade-offs between different measurements, to be able to choose the right metric for a given application. [62]

3.8.2 Inception Score

The Inception Score (IS) was introduced in [63] by Salimans et al as metric designed to evaluate the sample quality in a way that correlates with human judgement. They use an Inception v3 Network pre-trained on ImageNet, a dataset consisting of 1.2 million images with over 1000 classes. This powerful classification model outputs a vector of probabilities of the given image belonging to each class. The Inception score is calculated based on the output of the classification of the generated images $p(y|x)$ (also known as the conditional label distribution), with the idea that high-quality images should correspond to a confident prediction of a specific class.

The IS was designed with two metrics in mind: the sharpness and diversity of the generated images. Sharpness is assessed by conditional label distribution $p(y|x)$. If the image quality is high this distribution should have low entropy, meaning the model is confident about the class of the image. Diversity is measured by considering the marginal distribution over all generated images $p(y) = \int p(y|x)p_g(x)dx$. For a diverse set of images, this distribution should have high entropy, indicat-

ing that the generated images are varied and cover a wide range of classes. [63]

The Inception score function is defined as the KL divergence between these two distributions:

$$IS(G) = \exp\left(\mathbb{E}_{x \sim p_g} D_{KL}(p(y|x) \parallel p(y))\right) \quad (80)$$

With an added exponential to ease comparison. This metric favours models that generate images that are both clear and diverse.

While Inception Score has been widely adopted and correlates well with human judgement, it comes with a fair share of criticisms and limitations. In [61] Barrat et al identifies several major shortcomings of the Inception Score. The major ones being:

Sensitivity to weights. Due to the inherent randomness in the training procedure, different training runs can produce in different network weights, with minimal effect to the classification accuracy. The authors calculated the Inception Score using different implementations of the Inception network, all with almost identical classification accuracies. They found major discrepancies when comparing the resultant Inception scores. The authors suggest that these inconsistencies in metric it might be enough to put into question the validity of some “state-of-the-art” models evaluated by this metric.

The score being calculated on too small of a set as the 5000 samples used at a time are not enough to cover the 1000 classes of the dataset. This separation of the 50k samples into chunks of 5k, also introduces an unnecessary variable capable of changing the final score.

The usage on generative models trained on different datasets from the one the Inception network was trained on is a major flaw that leads to misleading results. The authors found (when using a inception network trained on ImageNet) that the average entropy of the conditional distribution $p(y|x)$ conditioned on an image from the CIFAR dataset is closer to the average entropy of the conditional distribution conditioned on a uniformly random image than to one conditioned on the ImageNet validation set by a sizable margin. The authors also point out the major discrepancy in classes between different datasets, such as CIFAR-10 (with 10 classes) and ImageNet (with 1000) leading to the most predicted classes by the ImageNet trained model to be quite confusing and specific, not matching the classes seen in CIFAR.

Amongst other flaws, such as examples in literature of algorithms using inception score to optimize early stopping, hyperparameter tuning or model architectures, leading to the models optimizing to generate adversarial samples, not reporting metrics on overfitting alongside the IS, etc.

Despite so many flaws, the Inception Score can still be a useful tool in generative model evaluation, particularly when it’s used in combination with other metrics, and with cautious interpretation of its results.

3.8.3 Fréchet Inception Distance

The Fréchet Inception Distance was introduced in 2018 by Heusel et al [64] as an alternative to the Inception Score, with the goal of providing a more robust and reliable metric for evaluating generative models that better aligns with human judgment. They design this metric based on the assumption that the equality $p(.) = p_w(.)$ holds except for a non-measurable set if and only if $\int p(.)f(x)dx = \int p_w(.)f(x)dx$ for a basis $f(.)$ that encompasses the function space where p and p_w exist. By mapping these distributions on to a high-level feature space they assume these distributions can be approximated as multivariate gaussians. The feature map is obtained by passing each real and generated sample into an Inception-V3 classifier network trained on ImageNet using the output from the penultimate layer of the network as embeddings for the comparison. [77] The Fréchet distance is then applied to these real and generated embeddings using the following formula:

$$FID = \|\mu_r - \mu_g\|_2^2 + \text{tr} \left(\Sigma_r + \Sigma_g - 2\sqrt{\Sigma_r \Sigma_g} \right) \quad (81)$$

Where μ_r and Σ_r denote the mean and covariance of the real embedding distribution and μ_g and Σ_g represent the same for the generated sample embedding distribution. The matrix trace operator tr sums the diagonal elements of the matrices. This approach offers several advantages over the Inception score. FID takes the dataset the generator model was trained on into account, allowing it to be used with unlabelled datasets. It correlates better with human judgment and is more sensitive to different types of sample corruption. Unlike IS, it measures both quality and diversity of a dataset and is also able to detect intra-class mode collapse. [79]

However, FID also has several drawbacks. Just like the IS, relying on an Inception-V3 network pretrained on ImageNet makes it biased against datasets that differ significantly from it, which could pose a problem when evaluating Mel-spectrogram images. The FID score can vary a lot depending on the specific implementation, number of samples used and even small image processing details, making it a less reliable metric for cross-model comparisons [77]. Having one single value measuring both diversity and sample quality can obscure important details about the models, having been found to have a blind spot when it came to sample quality [79], it can also be the case that improving diversity can worsen sample quality and vice versa [77]. The assumption that the Inception model's high-level feature space has any perceptual relevance is pretty flawed as improving FID does not always mean a perceptual improvement in the generated samples.

3.8.4 Fréchet Audio Distance

The Fréchet Audio Distance (FAD) was introduced by Kilgour et al in [65] as an adaptation of FID to evaluate generative and enhancement models in the audio and music domains. The key differ-

ence being the replacement of the Inception-V3 network trained on ImageNet, to a model trained with an audio classification objective. Specifically, Kilgour et al used the VGGish network, which is based on the VGG image recognition network but instead trained as an audio classifier on a dataset of YouTube videos with over 3000 classes. The VGGish processes 1 second audio clips in a mel-spectrogram format and generates embeddings from both real and generated audio samples. The Fréchet distance is then calculated between those two embedding distributions to evaluate the quality of the generative or enhancement models.

This approach is commonly used for the evaluation of music generation models along with 3 other embedding models OpenL3, TRILL and MuLan. A common approach being to use the MusicCaps dataset (which consists of 10 second clips of music-related segments from YouTube videos), as the reference dataset (real audio distribution) to measure it's Fréchet distance from a distribution of generated music samples.

Gui et al in [66] identified several limitations with these commonly used FAD implementations. Finding that the many of the commonly used embedding models, including VGGish, do not reliably predict the perceptual quality of generated music. They also highlighted the inconsistencies in literature due to a lack of standardized FAD configurations (different models, sample sizes, reference datasets, embeddings...) making comparisons unfeasible. And one of the most commonly used reference dataset MusicCaps to have a large amount of poor audio quality samples and even some non-music samples.

To address these issues, these authors introduced several improvements to FAD, along with new embedding models and reference sets, which they thoroughly evaluate their performance in assessing acoustic and musical quality of data samples. With this providing a more reliable and standardized metric for generative music evaluation.

They introduce two new datasets to be used as reference sets, FMA-Pop consisting of 4230 songs from the Free Music Archive, evenly selected from 163 genres with varying degrees of sound quality; and MusCC consisting of 49 songs from the msdb18 dataset and all 240 songs from the MoisesDB dataset, all professionally mixed songs with studio-quality recordings. Both these datasets improve FAD's correlation with perceptual judgments and offer more reliable FAD scores particularly when evaluating generative music models across different musical genres.

They introduced and tested multiple embedding models: CLAP and LAION CLAP (audio and music) being joint audio-text embedding models, MERT being a self-supervised audio embedding model, CDPAM being an audio similarity model and EnCodec, EnCodec48k [76] and Descript Audio Codec (DAC) being low-rate audio codec models. They also used VGGish as a baseline comparison for their experimentation and evaluations.

They found that sample size had a big impact on FAD scores, with FAD decreasing when sample size was increased, at varying rates depending on the embedding model and generative set. To mitigate this bias, they introduce the FAD_{∞} based on the FID_{∞} proposed by Chong et al in [80], wherein they extrapolate the unbiased FAD score estimate at an infinite sample size using linear regression.

They introduce a way to compute FAD scores on a per-sample basis, helping identify outliers that could otherwise skew the overall evaluation and to identify certain biases with particular embedding models or reference sets such as the VGGish model having a bias against bagpipes music and the MusCC set having a bias towards western music genres.

Through a series of tests and evaluations (acoustic sensitivity tests, music quality prediction tests, outlier detection and examination and human assessment) they compared the different embedding models across the 3 different reference sets each. The results highlighted performance differences among models and their strengths and weaknesses in capturing various acoustic and musical qualities along with certain biases found.

In this paper they also introduced the FAD toolkit, a convenient way to evaluate your audio generative models using the techniques, models and reference sets introduced in this paper and allowing the user to create their own embeddings.

3.8.5 Human Assessment

4 IMPLEMENTATION

In this section, I present a comprehensive overview of the implementation process for mel-spectrogram audio generation using diffusion models. The datasets used were the Speech Commands Zero Through Nine (SC09) Dataset, which is a subset of the Speech Commands dataset [68] consisting only of spoken digits “zero” through “nine”, used to train a conditional and an unconditional U-Net[58] models; and a subset of the MAESTRO Dataset [69] consisting only of 20 hours of randomly selected wave files used to train an unconditional U-Net model. Training was done using DDPM’s generation process [17], whilst using DDIM [55] for tasks requiring efficient sampling. To simplify and expedite model training, I used the diffusers library as the foundation for my code. The models were evaluated using the Fréchet Audio Distance (FAD) metric proposed in [66] by Gui et al, using the Encodec[76] model to compute the embeddings from the baseline dataset and the set of generated audio. All model training, sampling and code execution was done on Google Colab, using the A100 GPU. The code along with model training and evaluation notebooks can be found here: <https://github.com/sebascorreia/music-diffusion>

4.1 Audio Representation

The audio representation chosen was Mel-spectrogram images, as they provided a less computational and memory intensive alternative to raw audio, while keeping enough information for accurate modelling. This is quite a popular audio representation in literature, as it is a much cheaper and efficient approach to train and sample audio generation models being more scalable in time length. [7]

The code for the audio pre-processing came from the `teticio/audiodiffusion` repository [72], which used to be part of the `diffusers` library, partly altered to fit the needs of this project. It works by taking a folder of audio files, split them up or extend them timewise to fit the desired image resolution, converts the audio slices into Mel-spectrograms using the `Mel-spectrogram converter` function from the `librosa` library; converts the spectrogram to the logarithmic scale; then saves them as gray-scale images into a dataset which also includes the audio slice `.wav` file, the file path, slice number and class label if available.

For the `Maestro` dataset I kept the default parameters to generate the spectrogram from `teticio/audiodiffusion`, which were, 256x256 image resolution, hop length 512, sample rate of 22050, FFT window size of 2048 samples, decibel range of 80 db; for the `SC09` dataset I used the roughly the same metrics used by Donahue et al when training `SpecGAN` [3]: 128x128 image resolution, hop length 128, sample rate of 16k, FFT window size of 1024 samples, decibel range of 80 db.

This provided a good enough frequency resolution where the sound quality wouldn't be too deteriorated and a good enough temporal resolution the image represented about 5.96 second slices in the `Maestro` dataset and 1 second in the `SC09`.

To convert a spectrogram back to audio the `image_to_audio` function is used. This converts the image back into byte data, then the data is scaled back from decibels to power, and the `mel_to_audio` function from the `librosa` package is used to reconvert it to a raw audio array. The `mel_to_audio` function uses the Griffin-Lim algorithm to convert the spectrogram into audio. As this is a lossy spectrogram inverter it slightly deteriorates the audio quality. This might negatively impact evaluation.

4.2 Datasets

The `SC09` dataset is a subset of the `Speech Commands` dataset, consisting exclusively of 38,916 audio recordings of spoken digits from "zero" to "nine." Each recording is approximately one second long and was collected in diverse acoustic environments with multiple speakers, contributing to the dataset's variability. This diversity makes `SC09` an excellent choice for training and evaluating both conditional and unconditional models, as it challenges the model to generalize across different voices and recording conditions. The labelled nature of `SC09` simplifies the evaluation

process, enabling straightforward assessment of the model's accuracy in generating recognizable digits. The dataset can be accessed [here](#).

The MAESTRO dataset is a large-scale collection of over 200 hours of virtuoso piano performances, accompanied by corresponding MIDI files. This dataset is highly regarded in the music generation community due to its high-quality recordings and detailed MIDI annotations, which provide rich information for model training. For this project, I selected a random subset of about 20 hours of wav files and pre-processed them into a dataset with about 12k audio slices and their respective Mel-spectrogram images. This dataset can be accessed [here](#).

I selected these two datasets not only for their prominence in existing research but also for their complementary characteristics. SC09's labelled, spoken digit recordings allow for clear-cut, objective evaluation. In contrast, the Maestro dataset, being a much more complex higher-resolution, unlabelled music dataset, presents a more subjective challenge in evaluation, as the quality of the generated outputs may vary depending on individual musical interpretation. [3] The higher resolution allows for longer time frames allowing for evaluation of the model's temporal coherence. The datasets were separated into test and train sets with a 20% data split.

4.3 Architecture

The three models used have roughly the same architecture. The diffusers library was used to define the models using the UNet2D model with the default block configurations, using the architecture from their training tutorial [73] as it had been proved to be quite good at mel-spectrogram generation by the teticio/audio-diffusion repository [72]. They are U-net models [58] as are most diffusion model, they take in a single input channel accepting only gray-scale images and output a grayscale image with a single output channel of the desired resolution which for Maestro is 256x256 and for SC09 is 128x128.

4.3.1 Blocks

Each down-sampling and up-sampling block contains two layers of residual blocks. This residual block composed of two group normalization layer plus convolutional layer pairs, followed by a non-linearity operation using the SiLU function. For the down-sampling blocks a convolutional down-sampling operation is added after each residual block up until the final block. The same is done in the upsampling layers but with upsampling operations. Skip connections are used when moving from one block to a block with a different amount of channels. These shortcuts help preserve the low-level information from earlier blocks by combining it with the high-level information of the later blocks during the upsampling, they also mitigate the vanishing gradient problem. [59,60]

The attention blocks add an attention layer with 8-dimensional attention heads after each residual

block. An up or down sampling layer is also added to the block after the attention layer. This attention layer can provide localized information allowing the network to attune to particular features in the spectrogram. When provided with class information the attention gate applies a function to weigh the feature map according to each class, enabling the model to focus on the particular class. [59]

The model consists of four down-blocks, followed by an attention down-block and another down-block, this configuration is mirrored by their up-block versions in the decoder. The encoder part of the U-Net gradually reduces the spatial dimensions of the input while increasing the dimensions of the feature map. This is mirrored by the decoder part of the U-Net to get an image of the same dimensions as the input from the deeply convoluted latent code. The encoder's block output channel dimensions following this set [128, 128, 256, 256, 512, 512], this set is mirrored for the decoder.

The conditional model takes in the number of class embeddings as a parameter at initialization, taking in the labels of the input data only in the forward process. I had to make my own DDIM Pipeline class inheriting the DDIM pipeline from `diffusers` as it did not allow for simple class embeddings. Though I did not have to change much besides adding `class_labels` to the pipeline's initialization and forward processes.

4.4 Training

4.4.1 Training Setup

Training was performed using the DDPM scheduler and pipeline from the `diffusers` library, with a linear variance schedule where β_t going from 0.0001 to 0.02 over 1000 timesteps, following the approach outlined by Ho et al [17]. The optimizer and learning rate scheduler are kept the same as the configuration used in the training tutorial. Using AdamW as the optimizer and learning rate with a cosine schedule with 500 warmup steps. The mel-spectrogram images were normalized to a mean and variance of 0.5 rescaling the pixel values to be between [-1,1], then turned into tensors and the labels were mapped into integers. The Accelerate library [74] was used to optimize training allowing for effective parallelization and handling of model parameters.

The models are trained on an A100 GPU in Google Colab, as lower-tier GPUs were encountering memory issues when processing the large amount of data from the Maestro dataset. Although, I attempted to use Mixed Precision to reduce the memory consumption, I was unable to successfully integrate it into the training process. This constrained my ability to explore and finetune the models, as training with the A100 is very costly and my budget was limited. As a result, I decided to keep hyperparameters at the default values provided by the training tutorial [73].

An Exponential Moving Average (EMA) of the model parameters was used throughout training, to stabilize learning and improve performance over longer training sessions. I referenced the audio-

diffusion training script [72] to implement EMA into my own script. I used the EMA hyperparameters recommended by the diffusers library for long form training of more than a million steps, which are 2/3 ema power, ema warm up enabled, and a decay factor of 0.9999.

4.4.2 The Training Script

My training script was based on the training tutorial provided by the diffusers library [73] with some alterations to fit my models. Training is done by first loading and transforming the dataset, defining a scheduler, a model and a pipeline, declaring an optimizer a learning rate scheduler and an EMA model, and preparing the accelerator. If the start epoch is not set to zero the script skips all the corresponding steps until the right epoch is reached, this is used to continue training on pre-trained models.

With everything ready the model is set to train, and the script follows the DDPM training algorithm from [17]. Clean images along with their respective class labels (if training the conditional model) are sampled from each batch. Noise is generated using the shape of the images, and the scheduler, noise is used to add this noise to the images at randomly sampled timesteps. The model predicts the noise added to the image at the corresponding timestep, and the loss function is calculated by taking the Mean Squared Error (MSE) between the predicted and the actual noise.

I trained the SC09 models for about 200 epochs and the Maestro model for 300 epochs. Checkpoints were saved every 10 epochs. A batch size of 64 was used for the SC09 models, while the Maestro model was limited to a batch size of 16 as the larger resolution was too memory intensive for the GPU, not allowing for larger batches.

Although I implemented the option to evaluate the model during training by generating images and calculating the model's Fréchet Audio Distance (FAD), this was ultimately not used it due to the long time needed to calculate the FAD and the need to keep training as efficient as possible.

4.5 Evaluation

The models were evaluated using quantitative and qualitative metrics. These were the Fréchet Audio Distance (FAD), the reconstruction Mean Squared Error (MSE), interpolation and latent space sampling. All the models were evaluated using the DDIM scheduler with 50 timesteps with $\eta = 0$ for efficient sampling and performance without sacrificing sample quality.

4.5.1 Latent space sampling

Latent space sampling refers to the ability of the model to generate new audio samples by sampling from the learned latent space. This is done by sampling random gaussian noise and iteratively denoising it to produce a coherent output, in this case, Mel-spectrogram images which are then inverted back into audio. The process works as follows:

Within the DDIM pipeline, a tensor of random Gaussian noise is generated matching the desired shape of the Mel-spectrogram. This noise serves as the starting point in the latent space x_T . The scheduler generates a trajectory with 50 timesteps, which is a subsequence of the original 1000-step training trajectory. The random noise, timestep and class label (if provided) are passed into the U-Net model to predict the noise at the previous timestep $\epsilon_\theta(x_{t-1})$.

The predicted noise is then used by the scheduler to predict the mean of the image x_{t-1} , then using Eq. (77) the image itself is predicted. Since $\eta = 0$ no random noise is added as $\sigma_t = 0$, ensuring a deterministic and efficient sampling approach. This is repeated for the full sampling trajectory until the image is fully denoised. The result of this process is a fully generated Mel-spectrogram which is subsequently inverted into audio using the Griffin-Lim Algorithm.

I use the `Image_audio_generator.py` script to test the model's sampling. This allows for the selection of the pretrained model, the output directory, how many images to generate, scheduler type, the random noise seed and the eta value η used for the DDIM scheduler. For conditional models a specific label can be chosen to sample from. After getting the chosen pipeline, scheduler, model and Mel configurations, it sets the scheduler timesteps to the sampling trajectory and the generate function of the `evaluation.py` file is called. This function generates the chosen amount of spectrogram images in batches, then inverts them into audio using the `image_to_audio` function from the Mel class and saves them both on to the specified directory.

This script is available to use in the “Test.ipynb” notebook.

4.5.2 Reconstruction

Reconstruction is done by encoding a clean image through the model's noising process through a full sampling trajectory, then inputting that noise into the model's inference process to fully denoise the image. This process is only possible for DDIM models as the DDPM is too stochastic to generate an accurate reconstruction.

To evaluate the model in terms of reconstruction I implemented two scripts, one to do individual reconstruction, inspect the reconstructed images and audios and compare to the original images called `reconstruction.py` and another to calculate the reconstruction MSE over the entire test dataset.

4.5.2.1 Individual Reconstruction

The `reconstruction.py` script allows for either random or specific image selection from the test dataset, either specifying index, and/or label. Once the image is selected the specified model, pipeline, scheduler and appropriate Mel configurations are set and the reconstruction function from the `evaluation.py` file is called.

Here the image is normalized and processed into a tensor for model compatibility, then input into the noise function. This noise function was adapted from the teticio/audio-diffusion pipeline [72]. The noise function mimics the diffusion process, iteratively adding noise timestep by timestep over the full sampling trajectory until the image is destroyed. It uses an inversion of Eq. (77) to get x_t from x_{t-1} :

$$x_t = \sqrt{\alpha_t} \frac{(x_{t-1} - \sqrt{1 - \alpha_{t-1}} \epsilon_{\theta}^t)}{\sqrt{\alpha_{t-1}}} + \sqrt{1 - \alpha_t} \epsilon_{\theta}^t \quad (82)$$

The denoising function takes that noisy image, inputs it into the models forwards pass and predicts the next noise sample, which is then used to predict the image at the previous timestep using the same scheduler process as when sampling from the latent space. This is repeated until the image as been completely denoised. The deterministic process of the DDIM enables an accurate reconstruction of the original image.

After noising and denoising, the MSE between the original and reconstructed images is computed. The reconstructed image and the MSE result are returned to the script, where the image is post-processed from a tensor back into a grayscale image, this is inverted into audio using the Griffin-Lim algorithm, and both the image and the audio are saved into the output directory.

4.5.2.2 Reconstruction Evaluation

The `reconstruction_eval.py` reconstructs the entire test dataset in batches, calculating the average mean square difference between every image of the dataset and its respective reconstruction. A lower MSE indicates a better reconstruction performance, meaning that the models effectively captured the high-level features of the original image and successfully restored it to the original form.

4.5.3 Interpolation

Interpolation evaluates the model's ability to transition smoothly between two different samples of the latent space, generating intermediate mel-spectrograms that represent a gradual shift from one input to the other. This technique is used to evaluate how well the model has learned to structure the latent space.

The interpolation process begins by selecting two mel-spectrogram images from the test dataset. For the SC09 dataset it is possible to filter the dataset to pick out images of a specific class, even when using an unconditional model. These images are then noised using the Noise function detailed above, to encode them into the latent space. These generated latent space encodings are then interpolated using one of two methods:

Linear interpolation (LERP), which blends the two noise vectors in a linear fashion, where a weight λ controls the contribution of each image to the interpolated image:

$$x_{LERP} = (1 - \lambda) * x_1 + \lambda * x_2 \quad (83)$$

Where x_1 and x_2 and the encoded images.

Spherical Interpolation (SLERP), which is calculated by finding an angle between two vectors in the latent space and interpolating along the surface of a high-dimensional sphere. This ensures a smoother and more natural transition in the latent space. The SLERP algorithm used was adapted from [75].

The interpolated latent is then denoised using the denoise function mentioned above, and post-processed into an image, which results in a clean Mel-spectrogram with characteristics from both original spectrograms. The images are converted into audio and saved into the output directory.

This script is available to use in the “Test.ipynb” notebook.

4.5.4 Fréchet Audio Distance (FAD)

The Fréchet Audio Distance is a perceptual metric that compares the distributions of generated audio and the real audio samples in an embedding space, evaluating both the fidelity and diversity of the generated audio. This embedding space is created using a pre-trained model. The FAD metric captures high-level perceptual features by comparing the mean and covariance of embeddings for real and generated audio.

After careful consideration of the different embedding models, I was drawn to the EnCodec [76] model for my project due its great acoustic sensibilities and satisfactory musical quality evaluation compared to other models lacking joint audio-text embeddings. EnCodec being trained on music, speech and noisy speech samples being quite useful for my speech and music models. Not needing additional conditional information was a major reason in choosing it, as choosing a model like L-CLAP would require joint audio-text embeddings. While MERT provided better results when it came to musical quality evaluation, it lagged behind quite a lot when it came to acoustic evaluation, therefore I found EnCodec to be quite a better model for overall evaluation.

I decided to not use FMA-Pop or the other proposed reference sets and instead use FAD using the training datasets as reference sets against the samples generated by my models. This is done with FID in [57] on the improved DDPM introduced by Nichol et al. Whilst this metric might not align perfectly with perceptual quality evaluation it is still a useful metric to compare the distance between the generated audio sample distribution and the real audio sample distribution within a feature map generated to encode audio of both speech and music. The embedding model should introduce enough biases and inaccuracies having been trained with both music and speech in completely different datasets.

For this project, I adapted the FAD toolkit (FADTK) introduced by Gui et al in [66] to fit my train-

ing set up and hardware limitations.

4.5.4.1 FAD Implementation

The FAD script sets up the pipeline, scheduler, model and Mel configurations before calling the FAD function from `evaluation.py`. The FAD function starts by generating 5000 new Mel-spectrograms from the model’s latent space and converting them into audio using the Griffin-Lim Algorithm. This generation is done in batches of 128 for the SC09 dataset and 64 for the MAESTRO dataset, using the DDIM schedule with 50 timesteps for speedy generation while maintaining high sample quality.

Both the real and generated audio samples are passed through the pre-trained EnCoded model, which encodes the samples into a lower-dimensional embedding space. This allows the FAD metric to assess the high-level features of the audio that correspond to perceptual quality.

The FAD score is calculated by measuring the Fréchet Distance between the distributions of the real and generated embeddings using Eq.(81).

The adaptation of this toolkit was a lot harder than expected as the original implementation was very memory intensive. I had to take most of their functions and change them in order to preserve ram, whether by separating processing runs into batches or by changing the way files were stored into folders.

Starting with the `fad_script.py` which loads the pipeline, model, scheduler, and Mel configuration and invokes the FAD function from `evaluation`. Using roughly the same methods as in the `generate` function, the function first generates 5000 images, converts them into audio and stores them into folders containing 1000 audios each. This folder separation was added due to having problems retrieving files from folders with large amounts of files from Google Drive to Colab. It then takes all the audios from the dataset and stores them into separate folders with 1000 audios each. A model is chosen, and the `no_cache_embedding_files` function is called to generate the embedding files for both the real and the generated audios. This function from the `fad_functions.py` collects all the files that don’t have embeddings and using the multiprocessing library it parallelizes the CPU in order to convert the audios to the specific sampling the EnCodec model was trained for, and to generate the embeddings from those converted audios. Again, these embeddings are stored into separate folders as `.npy` files. I split up the files into batches as doing all files in one single run was crashing Google Colab’s system.

After the embeddings are processed and stored, the score function is then called. I had to make my own FAD class to change the file paths and to mitigate memory consumption. This score function calculates the mean and covariance matrix from each distribution, then finally calculates the Fréchet distance between these distributions. The result is printed into a text file to be stored within the

output directory.

This whole process takes about 20 hours to complete, I should have changed the fad toolkit functions to utilize the GPU instead of keeping their CPU multiprocessing as it perhaps would speed up the process, although I am not sure if the GPU could have handled that amount of memory as it has less than half the RAM the CPU has.

5 EXPERIMENTATION AND RESULTS

In this section I present the results of my experiments which evaluate the performance of the trained diffusion models in generating Mel-Spectrograms for audio reconstruction, interpolation and synthesis. The primary focus is to assess the models' ability to produce high-quality audio through both conditional and unconditional setups and to evaluate the robustness and structure of the latent spaces generated by these models.

These experiments were done using two distinct datasets: the SC09 dataset which consists of labelled spoken digits, and a subset of the MAESTRO dataset which consists of unstructured piano music. These datasets were chosen to test the models across a range of complexity, with SC09 being a larger smaller resolution of simpler labelled sounds, and Maestro providing a more complex, high-resolution dataset.

The evaluation is based on both quantitative and qualitative metrics. Quantitative evaluation metrics include the Fréchet Audio Distance (FAD) which measures the distance of the generated audio distribution to the distribution it was trained on within an embedding space, and the Mean Squared Error (MSE) which assesses the model's reconstruction ability and how well the model generalizes to an unseen sample distribution. For a qualitative assessment I will give my own subjective perceptual assessment of the samples both audio and images gathered from latent space sampling, reconstruction and interpolation.

5.1 Quantitative evaluation

Model	FAD	Reconstruction MSE
Unconditional Maestro U-Net	77.9169834249974	0.04880678653717041
Unconditional SC09 U-Net	140.26360231512714	0.22138308697059506
Conditional SC09 U-Net	117.93775246326459	0.09663992998052816

The quantitative evaluation of the models showed that the Unconditional Maestro U-net performing

the best under these metrics. Again, these metrics should not be taken as the end all proof that the model performs much better on the Maestro dataset than the SC09 dataset, it does help to orientate me into some conclusions. First, I will list the ways in which these models differs and why there might not be a fair comparison between them, and then I shall draw some conclusions about them in spite of these differences.

These models were trained using the same architecture despite their differences in resolution, sampling frequency, data amount, epochs and audio domains. This architecture has been shown to be effective when dealing with 256x 256 resolutions in both teticio/audio-diffusion and the diffusers training tutorial, but I could find no evidence of its performance on smaller resolutions images.

Examining the Mel-spectrograms images produced by both datasets it becomes quite clear just how different they are. The Maestro spectrograms appear more organized with a series of straight vertical and horizontal lines while the SC09 spectrograms seem to be much more complex with curving lines, and intricate shapes and shadows. This complexity could be attributed not only to the differing sonal properties of speech and music, but also differing configurations of each dataset. Although the core component of the Maestro dataset, as in the actual piano performances being played, is much more diverse than the 10 digits being spoken in the SC09 dataset, the varying voices, cadence, loudness, recording quality, etc. makes it quite a complex dataset, where the Maestro dataset has the virtually the same recording configurations, on the same piano, genre, all professionally performed, ensuring that the variability of the dataset can be almost narrowed down to just one factor, composition. The higher resolution of the Maestro spectrograms also allows the model to capture smaller details which might be blurred or too small to capture under lower resolutions.

The Maestro model was trained for 300 epochs whilst the SC09 models were only trained for 200, which might have given the Maestro model a better chance at converging, perhaps even overfitting to the train distribution which would most likely inflate the FAD score but be penalized when it came to reconstruction as it would make it worse at generalizing to the test distribution.

The lack of evaluation metrics at different epochs during training means I do not have enough information to analyse the possibility of overfitting and the model's convergence. Having a model such as SpecGAN act as a baseline would allow for a better understanding of how well these models are actually performing quantitatively, as the qualitative evaluation does tell us that the models are in fact capable of music and speech synthesis it is hard to quantify just how well without some baseline metrics.

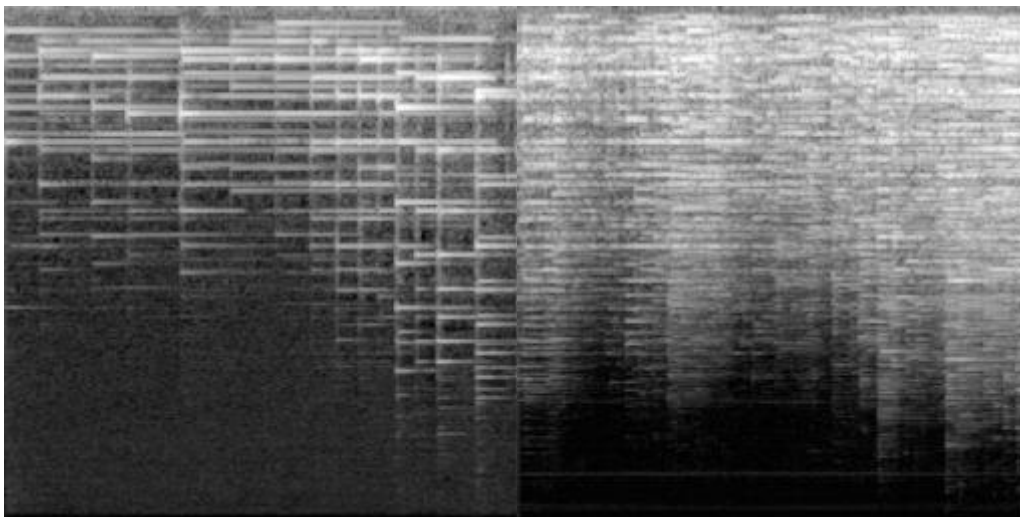
Qualitative evaluation:

As a poorly generated spoken digit might be more subjectively appealing than a poorly generated piano piece as it is much easier to match a small word than a 5 second piano piece. Therefore for practical purposes a speech model might not require as good as a performance as a music synthesis

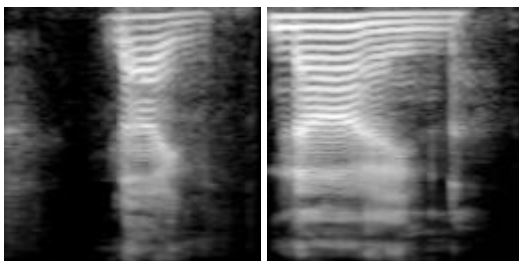
model, as long as the speech is clear.

5.2 Generation

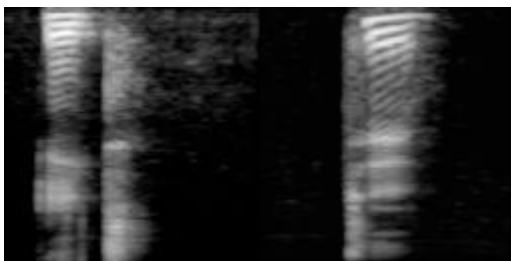
Unconditional Maestro Model:



Unconditional SC09 Model:

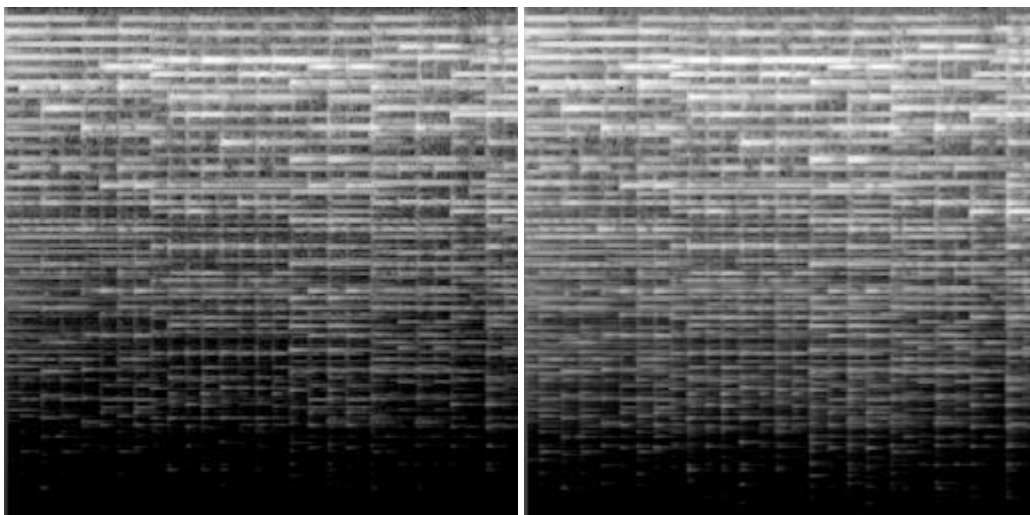


Conditional SC09 Model

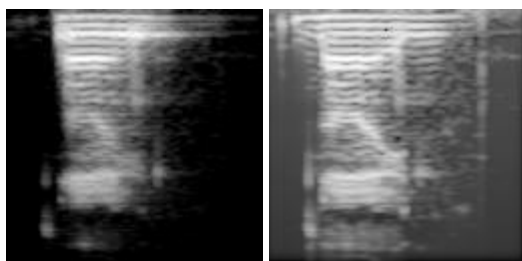


5.3 Reconstruction

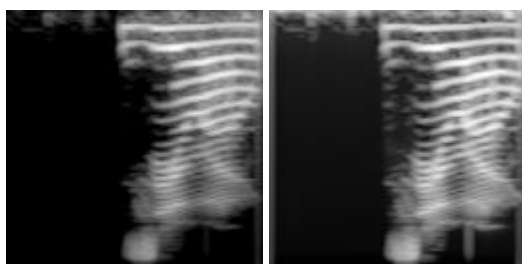
Unconditional Maestro Model:



Unconditional SC09 Model:

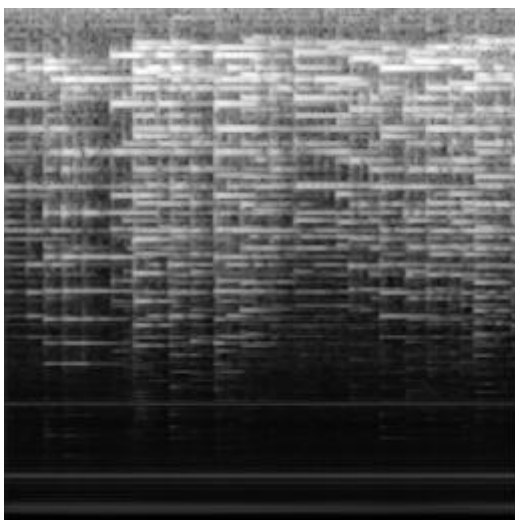
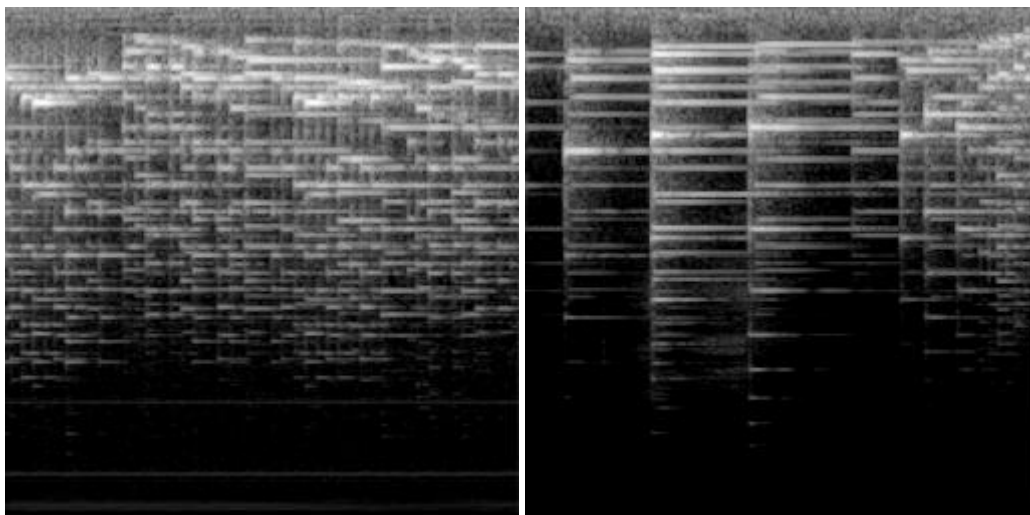


Conditional SC09 Model:

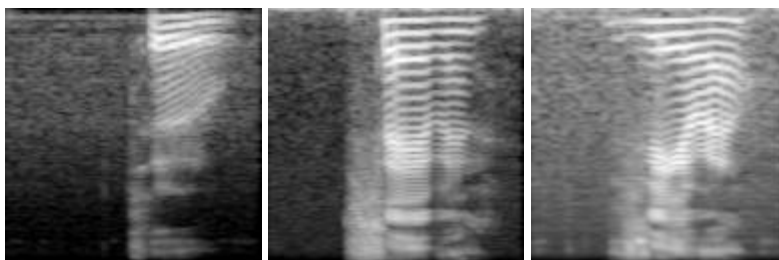


5.4 Interpolation

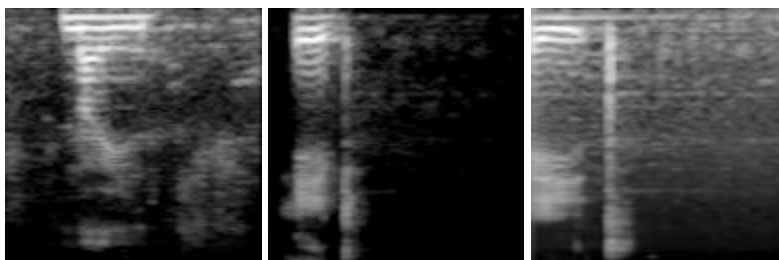
Unconditional Maestro Model:



Unconditional SC09 Model:



Conditional SC09 Model:



6 CONCLUSION

Diffusion models have shown quite promising results in the field of audio synthesis and

6.1 Future Work

REFERENCES

- [1] J.P. Briot, "From artificial neural networks to deep learning for music generation: history, concepts and trends," *Neural Computing and Applications*, vol. 33, no. 1, pp. 39-65, 2021.
- [2] A. van den Oord, S. Dieleman, H. Zen, K. Simonyan, O. Vinyals, A. Graves, N. Kalchbrenner, A. Senior, and K. Kavukcuoglu, "Wavenet: A generative model for raw audio," *arXiv preprint arXiv:1609.03499*, 2016.
- [3] C. Donahue, J. McAuley, and M. Puckette, "Adversarial audio synthesis," *arXiv preprint arXiv:1802.04208*, 2018.
- [4] F. Schneider, "Archisound: Audio generation with diffusion," *arXiv preprint arXiv:2301.13267*, 2023.
- [5] P. Dhariwal, H. Jun, C. Payne, J. W. Kim, A. Radford, and I. Sutskever, "Jukebox: A generative model for music," *arXiv preprint arXiv:2005.00341*, 2020.
- [6] Z. Borsos et al., "Audiolm: a language modeling approach to audio generation," *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, 2023.
- [7] Q. Huang et al., "Noise2music: Text-conditioned music generation with diffusion models," *arXiv preprint arXiv:2302.03917*, 2023.
- [8] A. Natsiou and S. O'Leary, "Audio representations for deep learning in sound synthesis: A review," in *2021 IEEE/ACS 18th International Conference on Computer Systems and Applications (AICCSA)*, pp. 1-8, 2021.
- [9] A. Marafioti, N. Holighaus and P. Majdak, "Time-Frequency Phase Retrieval for Audio—The Effect of Transform Parameters," in *IEEE Transactions on Signal Processing*, vol. 69, pp. 3585-3596, 2021, doi: 10.1109/TSP.2021.3088581.
- [10] A. Natsiou and S. O'Leary, "A sinusoidal signal reconstruction method for the inversion of the mel-spectrogram," in *2021 IEEE International Symposium on Multimedia (ISM)*, pp. 245-248, 2021.
- [11] G. Nierhaus, *Algorithmic Composition: Paradigms of Automated Music Generation*. Springer Science & Business Media, 2009.
- [12] T. Kitahara, "Music generation using Bayesian networks," in *Joint European Conference on Machine Learning and Knowledge Discovery in Databases*, pp. 368-372, Sept. 2017.
- [13] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735-1780, 1997.
- [14] Y. Yu, X. Si, C. Hu, and J. Zhang, "A review of recurrent neural networks: LSTM cells and network architectures," *Neural Computation*, vol. 31, no. 7, pp. 1235-1270, 2019.
- [15] S. Mangal, R. Modak, and P. Joshi, "LSTM based music generation system," *arXiv preprint arXiv:1908.01080*, 2019.

- [16] J. Sohl-Dickstein, E. Weiss, N. Maheswaranathan, and S. Ganguli, "Deep unsupervised learning using nonequilibrium thermodynamics," in International Conference on Machine Learning, pp. 2256-2265, PMLR, 2015.
- [17] J. Ho, A. Jain, and P. Abbeel, "Denoising diffusion probabilistic models," in Advances in Neural Information Processing Systems, vol. 33, pp. 6840-6851, 2020.
- [18] K. Preechakul, N. Chatthee, S. Wizadwongsa, and S. Suwajanakorn, "Diffusion autoencoders: Toward a meaningful and decodable representation," in Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, pp. 10619-10629, 2022.
- [19] V. Singh, "An in-depth guide to denoising diffusion probabilistic models – from theory to implementation," Mar. 6, 2023. <https://learnopencv.com/denoising-diffusion-probabilistic-models/#forward-diffusion-equation>
- [20] D. Bank, N. Koenigstein, and R. Giryes, 'Autoencoders,' in Machine Learning for Data Science Handbook: Data Mining and Knowledge Discovery Handbook, 2023, pp. 353-374.
- [21] U. Michelucci, 'An Introduction to Autoencoders,' arXiv preprint arXiv:2201.03898, 2022.
- [22] W. H. L. Pinaya, S. Vieira, R. Garcia-Dias, and A. Mechelli, "Autoencoders," in Machine Learning, Academic Press, 2020, pp. 193-208.
- [23] A. Ng, 'Sparse autoencoder,' in CS294A Lecture Notes, vol. 72, no. 2011, pp. 1-19, 2011.
- [24] A. Sarroff and M. A. Casey, 'Musical audio synthesis using autoencoding neural nets,' in Proc. International Society for Music Information Retrieval Conference (ISMIR2014), International Society for Music Information Retrieval, 2014.
- [25] J. Colonel, C. Curro, and S. Keene, "Improving neural net auto encoders for music synthesis," in Audio Engineering Society Convention 143, Audio Engineering Society, 2017.
- [26] S. Bengesi, H. El-Sayed, M. K. Sarker, Y. Houkpati, J. Irungu, and T. Oladunni, "Advancements in Generative AI: A Comprehensive Review of GANs, GPT, Autoencoders, Diffusion Model, and Transformers," *IEEE Access*, 2024.
- [27] Engel, J., Resnick, C., Roberts, A., Dieleman, S., Norouzi, M., Eck, D., & Simonyan, K. (2017, August). Neural audio synthesis of musical notes with WaveNet autoencoders. In Proceedings of the 34th International Conference on Machine Learning-Volume 70 (pp. 1068-1077). JMLR. org.
- [28] Roche, F., Hadjeris, G., & Pachet, F. (2018). Autoencoders for music sound modeling: A comparison of linear, shallow, deep, recurrent, and variational models. arXiv preprint arXiv:1806.04096.
- [29] I. Shafkat, "Intuitively Understanding Variational Autoencoders," Towards Data Science, Feb. 04, 2018. [Online]. Available: <https://towardsdatascience.com/intuitively-understanding-variational-autoencoders-1bfe67eb5daf>
- [30] T. Song, J. Sun, B. Chen, W. Peng and J. Song, "Latent Space Expanded Variational Autoencoder for Sentence Generation," in IEEE Access, vol. 7, pp. 144618-144627, 2019, doi:

10.1109/ACCESS.2019.2944630

- [31] D. P. Kingma and M. Welling, "Auto-Encoding Variational Bayes," arXiv preprint arXiv:1312.6114, 2013.
- [32] R. Savery and G. Weinberg, "Shimon the Robot Film Composer and DeepScore: An LSTM for Generation of Film Scores Based on Visual Analysis," arXiv preprint arXiv:2011.07953, 2020.
- [33] D. P. Kingma and M. Welling, "An introduction to variational autoencoders," *Foundations and Trends® in Machine Learning*, vol. 12, no. 4, pp. 307-392, 2019.
- [34] A. Caillon and P. Esling, "RAVE: A variational autoencoder for fast and high-quality neural audio synthesis," arXiv preprint arXiv:2111.05011, 2021.
- [35] L. Bottou, "Large-scale machine learning with stochastic gradient descent," in *Proceedings of COMPSTAT'2010: 19th International Conference on Computational Statistics*, Paris, France, August 22-27, 2010, Keynote, Invited and Contributed Papers, Physica-Verlag HD, 2010.
- [36] A. Asperti and M. Trentin, "Balancing Reconstruction Error and Kullback-Leibler Divergence in Variational Autoencoders," in *IEEE Access*, vol. 8, pp. 199440-199448, 2020, doi: 10.1109/ACCESS.2020.3034828.
- [37] B. M. Dillon, T. Plehn, C. Sauer, and P. Sorrenson, "Better latent spaces for better autoencoders," *SciPost Physics*, vol. 11, no. 3, p. 061, 2021.
- [38] I. Higgins, L. Matthey, A. Pal, C. P. Burgess, X. Glorot, M. M. Botvinick, et al., "beta-VAE: Learning basic visual concepts with a constrained variational framework," in *Proc. Int. Conf. Learn. Represent. (ICLR) (Poster)*, 2017, vol. 3.
- [39] C. Doersch, "Tutorial on variational autoencoders," arXiv preprint arXiv:1606.05908, 2016.
- [40] A. Alemi, B. Poole, I. Fischer, J. Dillon, R. A. Saurous, and K. Murphy, "Fixing a broken ELBO," in *International Conference on Machine Learning*, pp. 159-168, PMLR, 2018.
- [41] K. Kumar, R. Kumar, T. De Boissiere, L. Gestein, W. Z. Teoh, J. Sotelo, A. De Brebisson, Y. Bengio, and A. C. Courville, "Melgan: Generative adversarial networks for conditional waveform synthesis," *Advances in Neural Information Processing Systems*, vol. 32, 2019.
- [42] A. van den Oord and O. Vinyals, "Neural discrete representation learning," in *Advances in Neural Information Processing Systems*, vol. 30, 2017.
- [43] A. Razavi, A. van den Oord, and O. Vinyals, "Generating diverse high-fidelity images with vq-vae-2," in *Advances in Neural Information Processing Systems*, vol. 32, 2019.
- [44] A. Roy, A. Vaswani, A. Neelakantan, and N. Parmar, "Theory and experiments on vector quantized autoencoders," arXiv preprint arXiv:1805.11063, 2018.
- [45] I. Goodfellow, J. Pouget-Abadie, M. Mirza, B. Xu, D. Warde-Farley, S. Ozair, A. Courville, and Y. Bengio, "Generative adversarial networks," *Communications of the ACM*, vol. 63, no. 11, pp. 139-144, 2020.
- [46] M. Arjovsky and L. Bottou, "Towards principled methods for training generative adversarial networks," arXiv preprint arXiv:1701.04862, 2017.

- [47] Y. Yazici, C.-S. Foo, S. Winkler, K.-H. Yap, and V. Chandrasekhar, "Empirical analysis of overfitting and mode drop in gan training," in 2020 IEEE International Conference on Image Processing (ICIP), 2020, pp. 1651-1655.
- [48] M. Arjovsky, S. Chintala, and L. Bottou, "Wasserstein generative adversarial networks," in International Conference on Machine Learning, 2017, pp. 214-223.
- [49] A. Brock, J. Donahue, and K. Simonyan, "Large scale GAN training for high fidelity natural image synthesis," *arXiv preprint arXiv:1809.11096*, 2018.
- [50] A. Radford, L. Metz, and S. Chintala, "Unsupervised representation learning with deep convolutional generative adversarial networks," *arXiv preprint arXiv:1511.06434*, 2015.
- [51] I. Gulrajani, F. Ahmed, M. Arjovsky, V. Dumoulin, and A. C. Courville, "Improved training of Wasserstein GANs," in *Advances in Neural Information Processing Systems 30*, 2017.
- [52] R. Yamamoto, E. Song, and J.-M. Kim, "Parallel WaveGAN: A fast waveform generation model based on generative adversarial networks with multi-resolution spectrogram," in *ICASSP 2020-2020 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pp. 6199-6203, 2020.
- [53] X. Mao, Q. Li, H. Xie, R. Y. K. Lau, Z. Wang, and S. P. Smolley, "Least squares generative adversarial networks," in *Proceedings of the IEEE International Conference on Computer Vision*, pp. 2794-2802, 2017.
- [54] N. Mor, L. Wolf, A. Polyak, and Y. Taigman, "A universal music translation network," *arXiv preprint arXiv:1805.07848*, 2018.
- [55] J. Song, C. Meng, and S. Ermon, "Denoising diffusion implicit models," *arXiv preprint arXiv:2010.02502*, 2020.
- [56] C. Luo, "Understanding diffusion models: A unified perspective," *arXiv preprint, arXiv:2208.11970*, 2022.
- [57] A. Q. Nichol and P. Dhariwal, "Improved denoising diffusion probabilistic models," in *Proc. Int. Conf. Mach. Learn., PMLR*, 2021, pp. 8162-8171.
- [58] O. Ronneberger, P. Fischer, and T. Brox, "U-net: Convolutional networks for biomedical image segmentation," in *Medical Image Computing and Computer-Assisted Intervention—MICCAI 2015: 18th International Conference, Munich, Germany, October 5-9, 2015, Proceedings, Part III*, vol. 18, Springer International Publishing, 2015, pp. 234-241.
- [59] N. Siddique, S. Paheding, C. P. Elkin, and V. Devabhaktuni, "U-net and its variants for medical image segmentation: A review of theory and applications," *IEEE Access*, vol. 9, pp. 82031-82057, 2021.
- [60] F. Milletari, N. Navab, and S.-A. Ahmadi, "V-net: Fully convolutional neural networks for volumetric medical image segmentation," in *2016 Fourth International Conference on 3D Vision (3DV)*, IEEE, 2016, pp. 565-571.
- [61] S. Barratt and R. Sharma, "A note on the inception score," *arXiv preprint arXiv:1801.01973*,

- 2018.
- [62] L. Theis, A. van den Oord, and M. Bethge, "A note on the evaluation of generative models," arXiv preprint arXiv:1511.01844, 2015.
 - [63] T. Salimans, I. Goodfellow, W. Zaremba, V. Cheung, A. Radford, and X. Chen, "Improved techniques for training GANs," in *Advances in Neural Information Processing Systems* 29, 2016.
 - [64] M. Heusel, H. Ramsauer, T. Unterthiner, B. Nessler, and S. Hochreiter, "GANs trained by a two time-scale update rule converge to a local Nash equilibrium," in *Advances in Neural Information Processing Systems* 30, 2017.
 - [65] K. Kilgour, M. Zuluaga, D. Roblek, and M. Sharifi, "Fréchet audio distance: A metric for evaluating music enhancement algorithms," *arXiv preprint arXiv:1812.08466*, 2018.
 - [66] A. Gui, H. Gamper, S. Braun, and D. Emmanouilidou, "Adapting Fréchet audio distance for generative music evaluation," in *ICASSP 2024-2024 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pp. 1331-1335, 2024.
 - [67] C. Szegedy, V. Vanhoucke, S. Ioffe, J. Shlens, and Z. Wojna, "Rethinking the inception architecture for computer vision," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2016, pp. 2818-2826.
 - [68] P. Warden, "Speech commands: A dataset for limited-vocabulary speech recognition," arXiv preprint arXiv:1804.03209, 2018.
 - [69] C. Hawthorne, A. Stasyuk, A. Roberts, I. Simon, C.-Z. A. Huang, S. Dieleman, E. Elsen, J. Engel, and D. Eck, "Enabling factorized piano music modeling and generation with the MAESTRO dataset," arXiv preprint arXiv:1810.12247, 2018.
 - [70] P. von Platen, S. Patil, A. Lozhkov, P. Cuenca, N. Lambert, K. Rasul, M. Davaadorj, D. Nair, S. Paul, W. Berman, Y. Xu, S. Liu, and T. Wolf, "Diffusers: State-of-the-art diffusion models," GitHub repository, 2022. [Online]. Available: <https://github.com/huggingface/diffusers>
 - [71] R. Kumar, P. Seetharaman, A. Luebs, I. Kumar, and K. Kumar, "High-fidelity audio compression with improved RVQGAN," *Advances in Neural Information Processing Systems*, vol. 36, 2024.
 - [72] R. D. Smith, "Audio Diffusion," GitHub repository, <https://github.com/teticio/audio-diffusion>,
 - [73] Hugging Face, "Diffusers Training: Basic Training," *Hugging Face*, Sep. 2023. [Online]. Available: https://huggingface.co/docs/diffusers/tutorials/basic_training.
 - [74] S. Gugger, L. Debut, T. Wolf, P. Schmid, Z. Mueller, S. Mangrulkar, M. Sun, and B. Bossan, *Accelerate: Training and inference at scale made simple, efficient and adaptable*, 2022. [Online]. Available: <https://github.com/huggingface/accelerate>.
 - [75] Enzokro, A PyTorch SLERP implementation," *enzokro.dev*, Nov. 16, 2022. [Online]. Available: <https://enzokro.dev/blog/posts/2022-11-16-pytorch-slerp/>
 - [76] A. Défossez, J. Copet, G. Synnaeve, and Y. Adi, "High fidelity neural audio compression,"

arXiv preprint arXiv:2210.13438, 2022.

- [77] T. Kynkäänniemi, T. Karras, M. Aittala, T. Aila, and J. Lehtinen, “The Role of ImageNet Classes in Fréchet Inception Distance,” *arXiv.org*, 2022. <https://arxiv.org/abs/2203.06026>
- [78] M. S. M. Sajjadi, O. Bachem, M. Lucic, O. Bousquet, and S. Gelly, “Assessing generative models via precision and recall,” *Neural Information Processing Systems*, vol. 31, pp. 5228–5237, May 2018, [Online]. Available: <https://arxiv.org/pdf/1806.00035>
- [79] A. Borji, “Pros and cons of GAN evaluation measures: New developments,” *Computer Vision and Image Understanding*, vol. 215, p. 103329, Jan. 2022, doi: 10.1016/j.cviu.2021.103329.
- [80] M. J. Chong and D. Forsyth, "Effectively unbiased FID and Inception score and where to find them," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 6070-6079, 2020.

APPENDIX

A.1 Detailed Mathematical Concepts and derivations

A.1.1 Kullback-Leibler Divergence

The Kullback-Leibler Divergence is an asymmetrical distance measure between two probability distributions:

$$D_{KL}(P||Q) = \int p(x) \log \left(\frac{p(x)}{q(x)} \right) dx$$

A.2.1 Expectation of a random variable

The expected value of a random variable x is defined by the integral:

$$E[X] = \int_{-\infty}^{\infty} x f(x) dx$$

This represents the mean value that a random variable X is expected to take over a large number of observations.

The expectation with respect to a parameterized distribution :

$$E_{q_{\phi}(z)}[p_{\theta}(x|z)] = \int_{-\infty}^{\infty} q_{\phi}(z) p_{\theta}(x|z) dz$$